Unit:3 Process Management

Presented by:
Er. Binod Kumar Rajbhar
BE-Computer
ME- Information System Enginnering

What is a Process?

- A process is a program in execution.
- Process is not as same as program code but a lot more than it.
- A process is an 'active' entity as opposed to program which is considered to be a 'passive' entity.
- Attributes held by process include hardware state, memory, CPU etc.

Features of Process

- A process has a very limited lifespan
- They also generate one or more child processes, and they die like a human being.
- Like humans, even process has information like who is a parent when it is created, address space of allocated memory, security properties which includes ownership credentials and privileges.

What is a Program?

- A program is an executable file which contains a certain set of instructions written to complete the specific job on your computer.
- For example, Google browser chrome.exe is an executable file which stores a set of instructions written in it which allow you to view web pages.
- Programs are never stored on the primary memory in your computer. Instead, they are stored on a disk or secondary memory on your PC or laptop.
- They are read from the primary memory and executed by the kernel.

Features of Program

- A program is a passive entity. It stores a group of instructions to be executed.
- Various processes may be related to the same program.
- A user may run multiple programs where the operating systems simplify its internal programmed activities like memory management.
- The program can't perform any action without a run. It needs to be executed to realize the steps mentioned in it.
- The operating system allocates main memory to store programs instructions.

Process vs Program

BASIS FOR COMPARISON	PROGRAM	PROCESS
Basic	Program is a set of instruction.	When a program is execut it is known as process.
Nature	Passive	Active
Lifespan	Longer	Limited
Required resources	Program is stored on disk in some file and does not require any other resources.	Process holds resources su as CPU, memory address, disk, I/O etc.

Multiprogramming

- ✓ **Multiprogramming** is an ability of an operating system that executes more than one program using a single processor machine.
- ✓ More than one task or program or jobs are present inside the main memory at one point of time.
- ✓ **One real life example**: User can use **MS-Excel**, download apps, transfer data from one point to another point, Firefox or Google chrome browser, and more at a same time.

- ✓ Let’s P1 and P2 are two programs present in the main memory. The OS picks one program and starts executing it.
- ✓ During execution if the P1 program requires I/O operation, then the OS will simply switch over to P2 program.
- ✓ If the p2 program requires I/O then again it switches to P3 and so on.
- ✓ If there is no other program remaining after P3 then the CPU will pass its control back to the previous program.

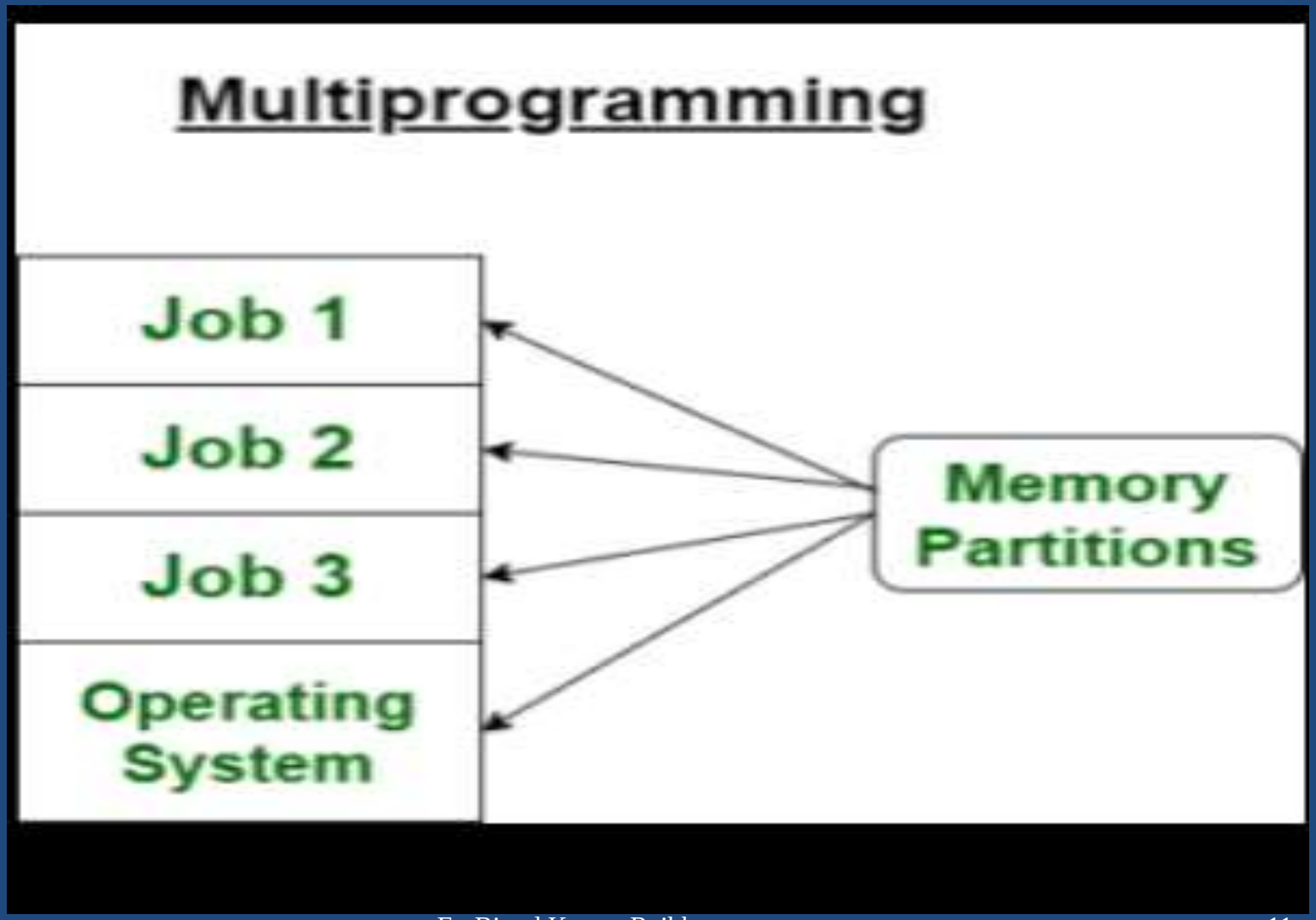
Advantages

- CPU utilization is high because the CPU is never goes to idle state.
- Memory utilization is efficient.
- Short time jobs are done fastest compare to long time jobs.
- Multiple users can use **multiprogramming system** at once.
- It can help to execute multiple tasks in single application at same time duration.

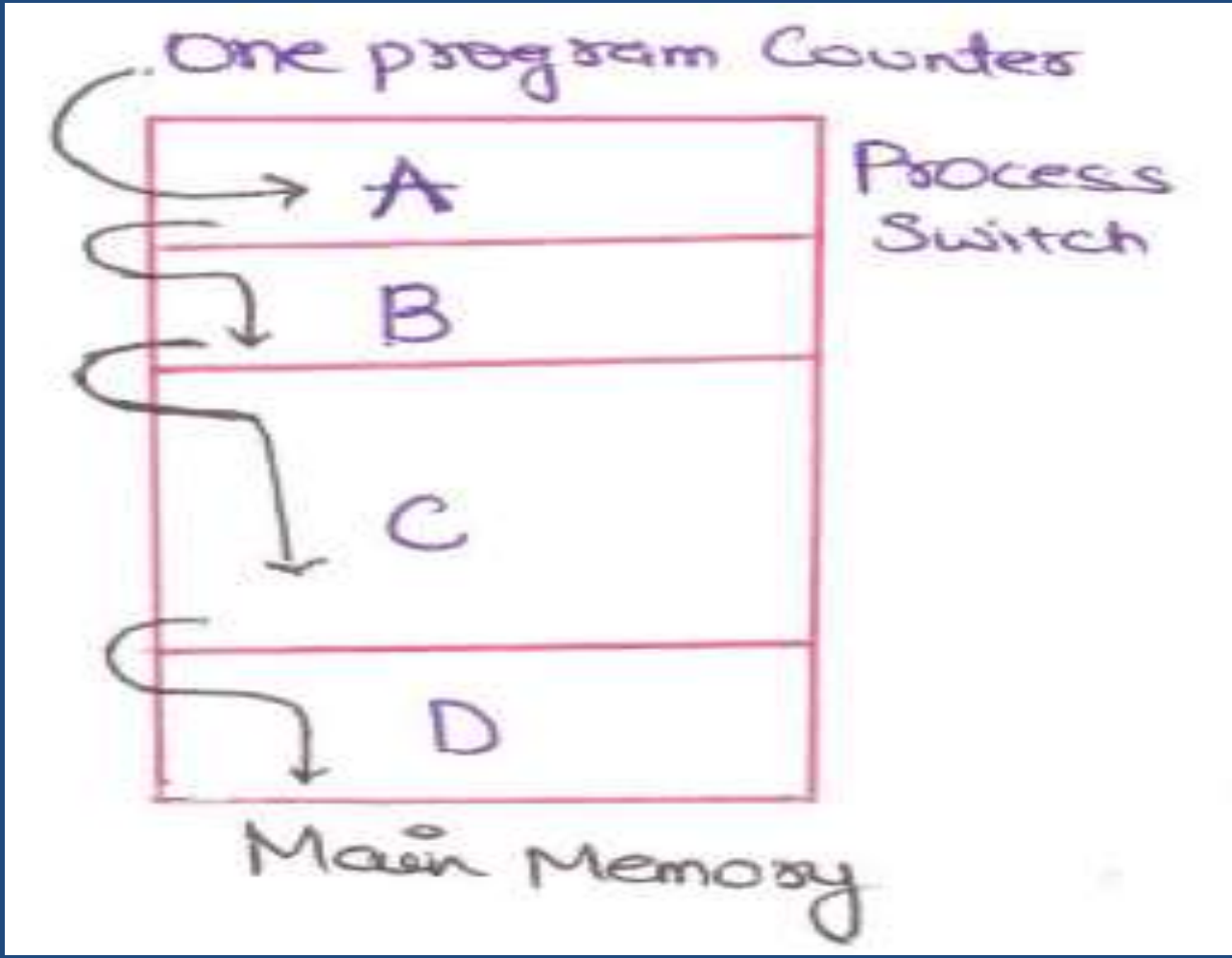
Disadvantages

- CPU scheduling is compulsory because lots of jobs are ready to run on CPU simultaneously.
- Memory management is required because all types of jobs are stored in the main memory.

Multiprogramming



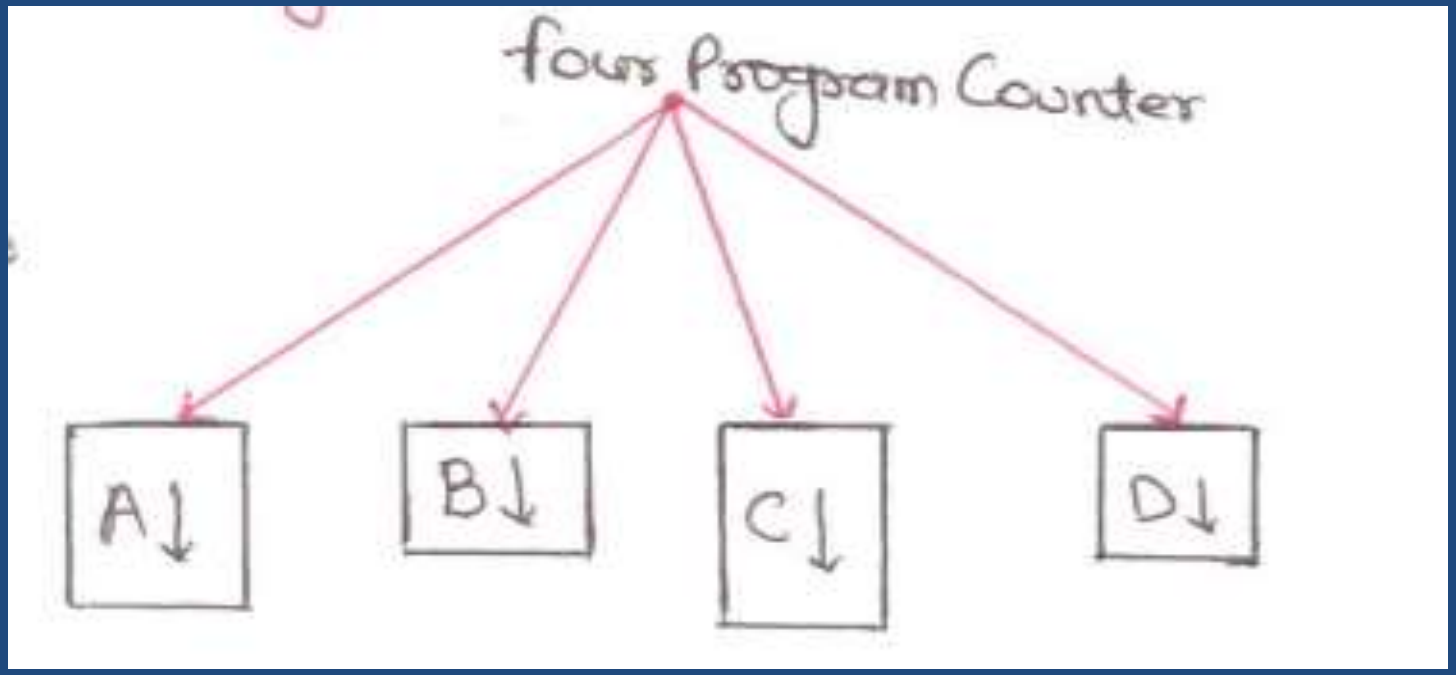
The Process Model



- Conceptually, every process has its own virtual CPU but in reality CPU switches back and forthfrom process to process, but to understand the system, it is much easier to think about a collection of processes running in (pseudo) parallel then to try to keep track of how the CPU switch is from program to program.
- This Rapid switching back and forth is called **"Multi-Programming**

Mainly three Types of Process Models are here

- Multiprogramming:



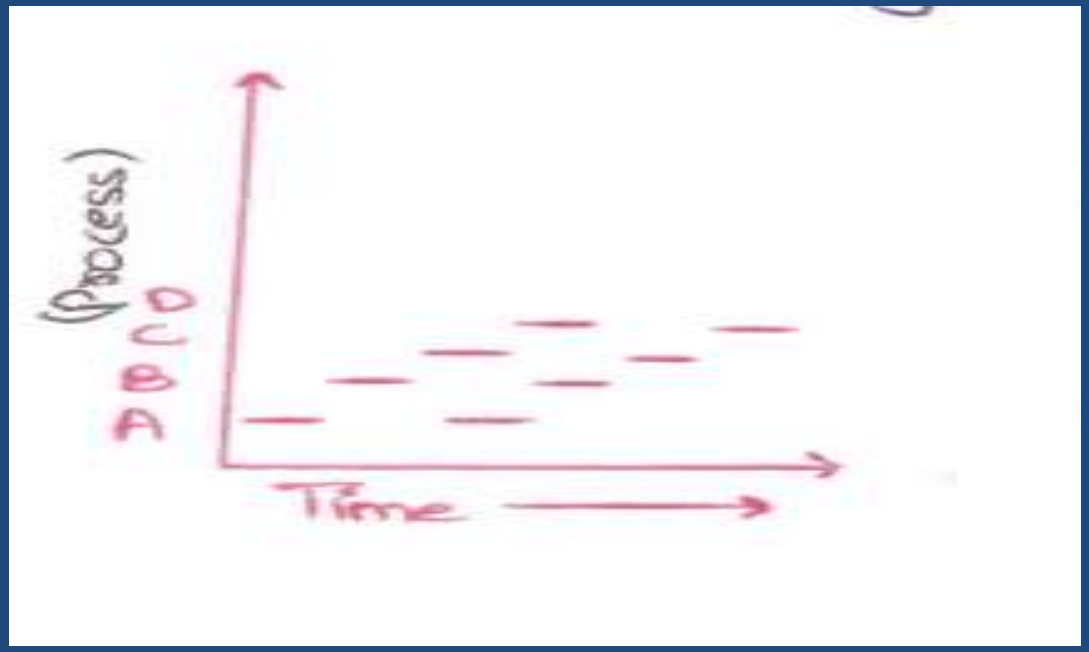
- In this there is 4 program in memory. We have single shared processor by all programs.
- There is only one program counter all programs in memory.
- In this program counter is initialized to execute the part of program A then with the help of process switch it transfer control to program B and execute some part of this.
- After that program counter for program C is active and execute some part of it and so onto program D.
- Then all the steps continuous may be randomly with the help of process switches until all program is not completed.

2) Multiprocessing:

- There is 4 processes, each with it own flow of control(that is it on logical program counter), and each when running independently of the other ones.
- There is only one physical program counter.
- So when each process runs, its logical program counter is loaded into the real program counter.
- When it is finished(for the time being), the physical program counter is saved in the process stored logical program counter in memory.

3) One Program At One Time

- At any given instant only one process run and other process after some interval of time.
- In other point of view all processes progress but only process actually runs at given instant of time

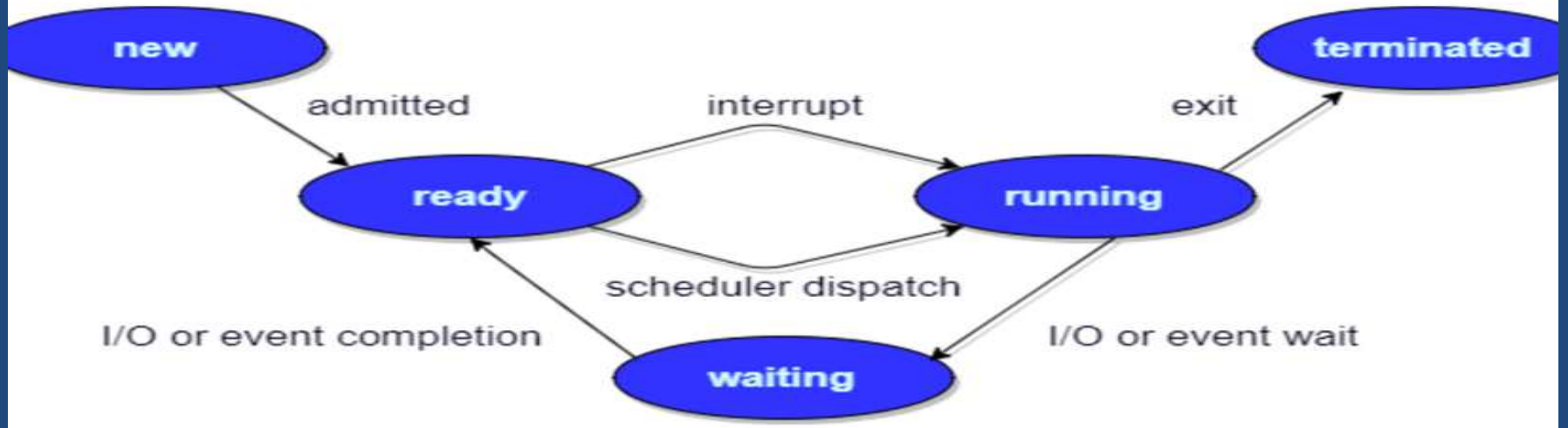


Different Process States

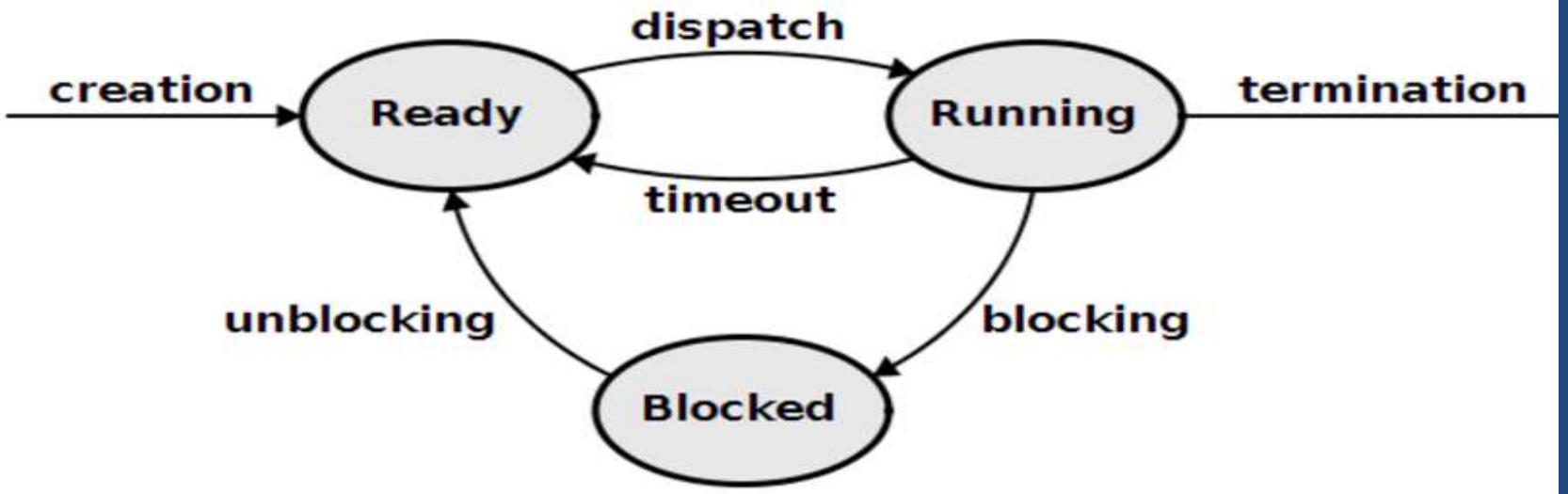
Processes in the operating system can be in any of the following states:

- NEW- The process is being created.
- READY- The process is waiting to be assigned to a processor.
- RUNNING- Instructions are being executed.
- WAITING- The process is waiting for some event to occur(such as an I/O completion or reception of a signal).
- TERMINATED- The process has finished execution

Process States



Process life cycle



Multiprocessing

- ✓ Multiprocessing is the ability of an operating system to execute more than one process simultaneously on a multi processor machine.
- ✓ A computer's capability to process more than one task simultaneously is called *multiprocessing*.
- ✓ A multiprocessing operating system is capable of running many programs simultaneously, and most modern network operating systems (NOSs) support multiprocessing. These operating systems include Windows NT, 2000, XP, and Unix.

Multitasking

- ✓ Multitasking is the ability of an operating system to execute more than one task simultaneously on a single processor machine.
- ✓ Multitasking system provides the interface for executing the multiple program tasks by single user at a same time on the one computer system.
- ✓ For example, any editing task can be performed while other programs are executing concurrently. Other example, user can open Gmail and Power Point same time

What is Process Control Block (PCB)?

- Process Control Block is a data structure that contains information of the process related to it.
- The process control block is also known as a task control block, entry of the process table, etc.
- It is very important for process management as the data structuring for processes is done in terms of the PCB.
- It also defines the current state of the operating system.

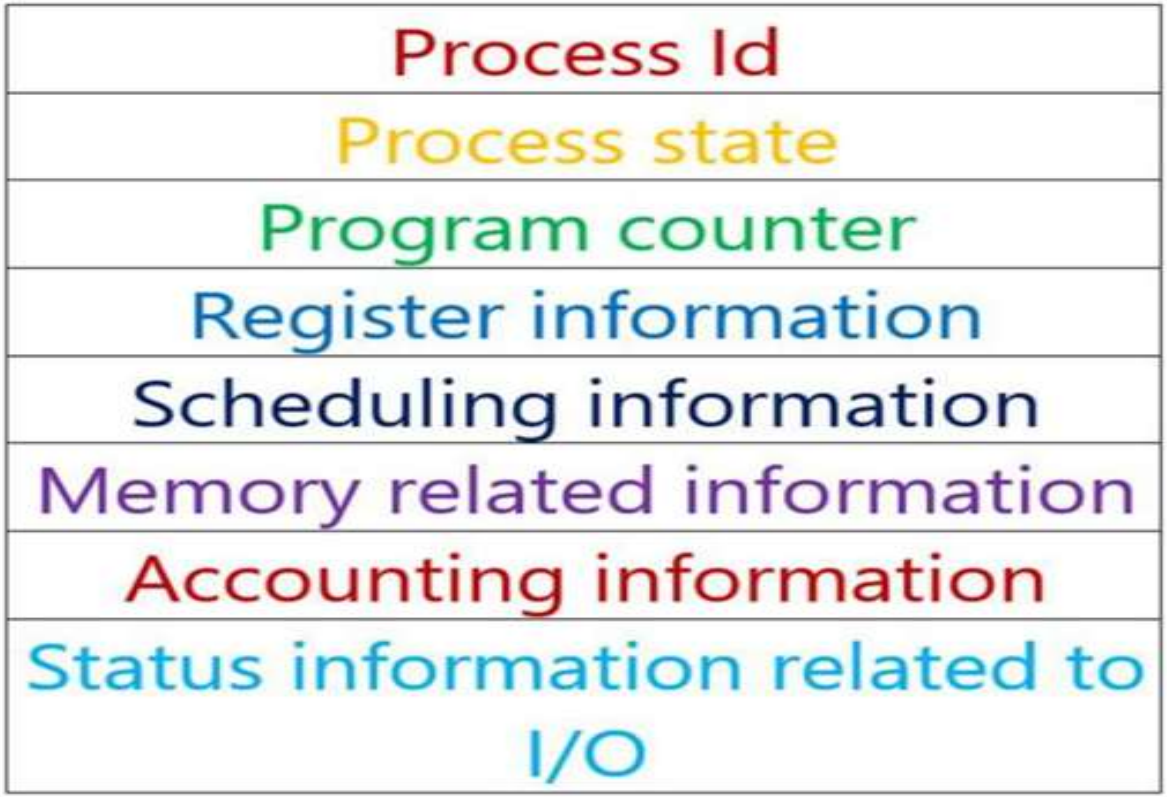
Role of PCB..

- Naming the process
- State of the process
- Resources allocated to the process
- Memory allocated to the process
- Scheduling information
- Input / output devices associated with process

Components of PCB

The following are the various components that are associated with the process control block PCB:

- **Process ID:**
- **Process State**
- **Program counter**
- **Register Information**
- **Scheduling information**
- **Memory related information**
- **Accounting information**
- **Status information related to input/output**


Process Id
Process state
Program counter
Register information
Scheduling information
Memory related information
Accounting information
Status information related to I/O

1. Process ID

- In computer system there are various process running simultaneously and each process has its unique ID.
- This Id helps system in scheduling the processes.
- This Id is provided by the process control block.
- In other words, it is an identification number that uniquely identifies the processes of computer system

2. Process state:

- As we know that the process state of any process can be New, running, waiting, executing, blocked, suspended, terminated.
- Process control block is used to define the process state of any process.
- In other words, process control block refers the states of the processes

3. Program counter:

- ✓ Program counter is used to point to the address of the next instruction to be executed in any process.
- ✓ This is also managed by the process control block.

4. Register Information:

- ✓ This information is comprising with the various registers, such as index and stack that are associated with the process.
- ✓ This information is also managed by the process control block

5. Scheduling information:

- ✓ Scheduling information is used to set the priority of different processes.
- ✓ This is very useful information which is set by the process control block.
- ✓ In computer system there were many processes running simultaneously and each process have its priority.
- ✓ The priority of primary feature of RAM is higher than other secondary features.
- ✓ Scheduling information is very useful in managing any computer system

6. Memory related information:

- ✓ This section of the process control block comprises of page and segment tables.
- ✓ It also stores the data contained in base and limit registers.

7. Accounting information:

- ✓ This section of process control block stores the details relate to central processing unit (CPU) utilization and execution time of a process.

8. Status information related to input / output:

- ✓ This section of process control block stores the details pertaining to resource utilization and file opened during the process execution

Process Table

- The operating system maintains a table called **process table**, which stores the process control blocks related to all the processes.
- The **process table** is a data structure maintained by the operating system to facilitate context switching and scheduling, and other activities.
- Each entry in the table, often called a **context block**, contains information about a process such as process name and state , priority , registers, and a semaphore it may be waiting on .
- The exact contents of a context block depends on the operating system.
- For instance, if the OS supports paging, then the context block contains an entry to the page table.

Definition's of Thread

- ✓ A thread is the smallest unit of processing that can be performed in an OS. In most modern operating systems, a thread exists within a process - that is, a single process may contain multiple threads.
- ✓ A thread is a path of execution within a process.
- ✓ A thread is a single sequence stream within in a process.
- ✓ Because have some of the properties of processes, they some times called lightweight processes

Thread

- You can imagine multitasking as something that allows processes to run concurrently, while multithreading allows sub-processes to run concurrently.
- When multiple threads are running concurrently, this is known as multithreading, which is similar to multitasking.
- One example of multithreading is downloading a video while playing it at the same time.

Advantages of Thread over Process

1. Responsiveness: If the process is divided into multiple threads, if one thread completes its execution, then its output can be immediately returned.

*2. **Faster context switch:*** Context switch time between threads is lower compared to process context switch. Process context switching requires more overhead from the CPU.

3. Effective utilization of multiprocessor system: If we have multiple threads in a single process, then we can schedule multiple threads on multiple processor. This will make process execution faster.

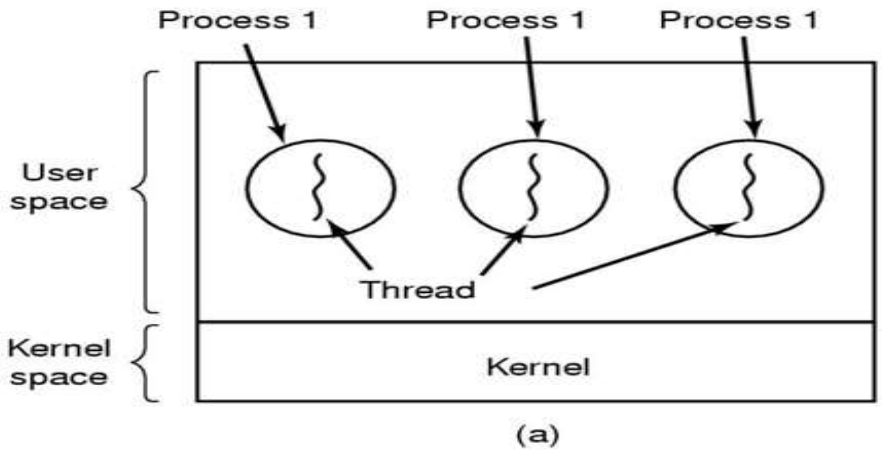
4. Resource sharing: Resources like code, data, and files can be shared among all threads within a process.

5. Communication: Communication between multiple threads is easier, as the threads shares common address space. while in process we have to follow some specific communication technique for communication between two process.

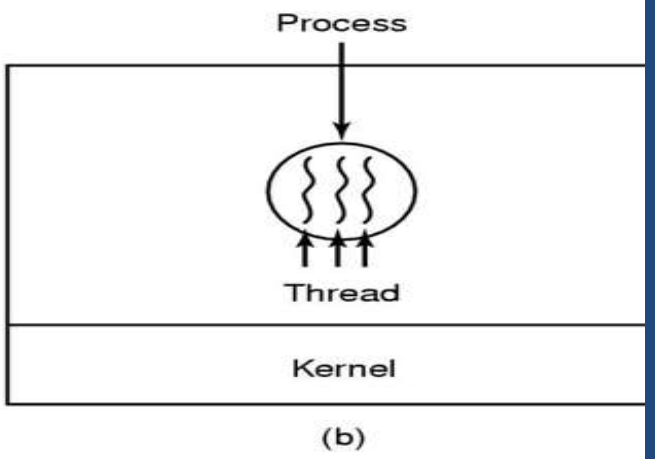
6. Enhanced throughput of the system: If a process is divided into multiple threads, and each thread function is considered as one job, then the number of jobs completed per unit of time is increased, thus increasing the throughput of the system.

Three processes each with one thread

One process with three threads



(a)



(b)

Difference between Process and Thread

BASIS FOR COMPARISON	PROCESS	THREAD
Basic	Program in execution.	Lightweight process or part it.
Memory sharing	Completely isolated and do not share memory.	Shares memory with each other.
Resource consumption	More	Less
Efficiency	Less efficient as compared to the process in the context of communication.	Enhances efficiency in the context of communication.
Time required for creation	More	Less
Context switching time	Takes more time.	Consumes less time.
Uncertain termination	Results in loss of process.	A thread can be reclaimed.
Time required for termination	More	

Definition	A process is a program under execution i.e an active program.	A thread is a lightweight process that can be managed independently by a scheduler
Context switching time	Processes require more time for context switching as they are more heavy.	Threads require less time for context switching as they are lighter than processes.
Memory Sharing	Processes are totally independent and don't share memory.	A thread may share some memory with its peer threads.
Communication	Communication between processes requires more time than between threads.	Communication between threads requires less time than between processes .
Blocked	If a process gets blocked, remaining processes can continue execution.	If a user level thread gets blocked, all of its peer threads also get blocked.
Resource Consumption	Processes require more resources than threads.	Threads generally need less resources than processes.
Dependency	Individual processes are independent of each other.	Threads are parts of a process and so are dependent.
Data and Code sharing	Processes have independent data and code segments.	A thread shares the data segment, code segment, files etc. with its peer threads.
Treatment by OS	All the different processes are treated separately by the operating system.	All user level peer threads are treated as a single task by the operating system.
Time for creation	Processes require more time for creation.	Threads require less time for creation.
Time for termination	Processes require more time for termination.	Threads require less time for termination.

Types of Thread

There are two types of threads:

- User Threads
- Kernel Threads

User Level thread (ULT)

- ✓ Is implemented in the user level library, they are not created using the system calls.
- ✓ Thread switching does not need to call OS and to cause interrupt to Kernel.
- ✓ Kernel doesn’t know about the user level thread and manages them as if they were single-threaded processes.

❖ **Advantages of ULT –**

- Can be implemented on an OS that doesn't support multithreading.
- Simple representation since thread has only program counter, register set, stack space.
- Simple to create since no intervention of kernel.
- Thread switching is fast since no OS calls need to be made.

❖ **Disadvantages of ULT –**

- No or less co-ordination among the threads and Kernel.
- If one thread causes a page fault, the entire process blocks.

Kernel Level Thread (KLT) –

- ✓ Kernel knows and manages the threads.
- ✓ Instead of thread table in each process, the kernel itself has thread table (a master one) that keeps track of all the threads in the system.
- ✓ In addition kernel also maintains the traditional process table to keep track of the processes.
- ✓ OS kernel provides system call to create and manage threads.

❖ **Advantages of KLT –**

- Since kernel has full knowledge about the threads in the system, scheduler may decide to give more time to processes having large number of threads.
- Good for applications that frequently block.

❖ **Disadvantages of KLT –**

- Slow and inefficient.
- It requires thread control block so it is an overhead.

Inter Process Communication

- ✓ IPC is a mechanism that allows the exchange of data between processes.
- ✓ Inter-process communication (IPC) is a set of techniques for the exchange of data among multiple threads in one or more processes.
- ✓ Processes may be running on one or more computers connected by a network.
- ✓ It is a set of programming interface which allow a programmer to coordinate activities among various program processes which can run concurrently in an operating system

Interprocess Communication

Processes executing concurrently in the operating system may be either independent processes or cooperating processes.

Independent processes - They cannot affect or be affected by the other processes executing in the system.

Cooperating processes - They can affect or be affected by the other processes executing in the system.

Any process that shares data with other processes is a cooperating process.

- ✓ Processes executing concurrently in the operating system may be either **independent process** **or** **co-operating process**

❖ **Independent process:**

- ✓ A process is independent if it can't affect or be affected by another process.

❖ **Co-operating Process:**

- ✓ A process is co-operating if it can affects other or be affected by the other process.
- ✓ Any process that shares data with other process is called co-operating process.

Reasons for providing an environment for process co-operation:

1.Information sharing:

- ✓ Several users may be interested to access the same piece of information(for instance a shared file).
- ✓ We must allow concurrent access to such information.

2.Computation Speedup:

- ✓ To run the task faster we must breakup tasks into sub-tasks.
- ✓ Such that each of them will be executing in parallel to other, this can be achieved if there are multiple processing elements.

3.Modularity:

- ✓ construct a system in a modular fashion which makes easier to deal with individual.

4.convenience:

- ✓ Even an individual user may work on many tasks at the same time. For instance, a user may be editing, printing, and compiling in parallel

Cooperating processes require an interprocess communication (IPC) mechanism that will allow them to exchange data and information.

There are two fundamental models of interprocess communication:

(1) Shared memory

(2) Message passing

- In the shared-memory model, a region of memory that is shared by cooperating processes is established.
  Processes can then exchange information by reading and writing data to the shared region.

- In the message passing model, communication takes place by means of messages exchanged between the cooperating processes.

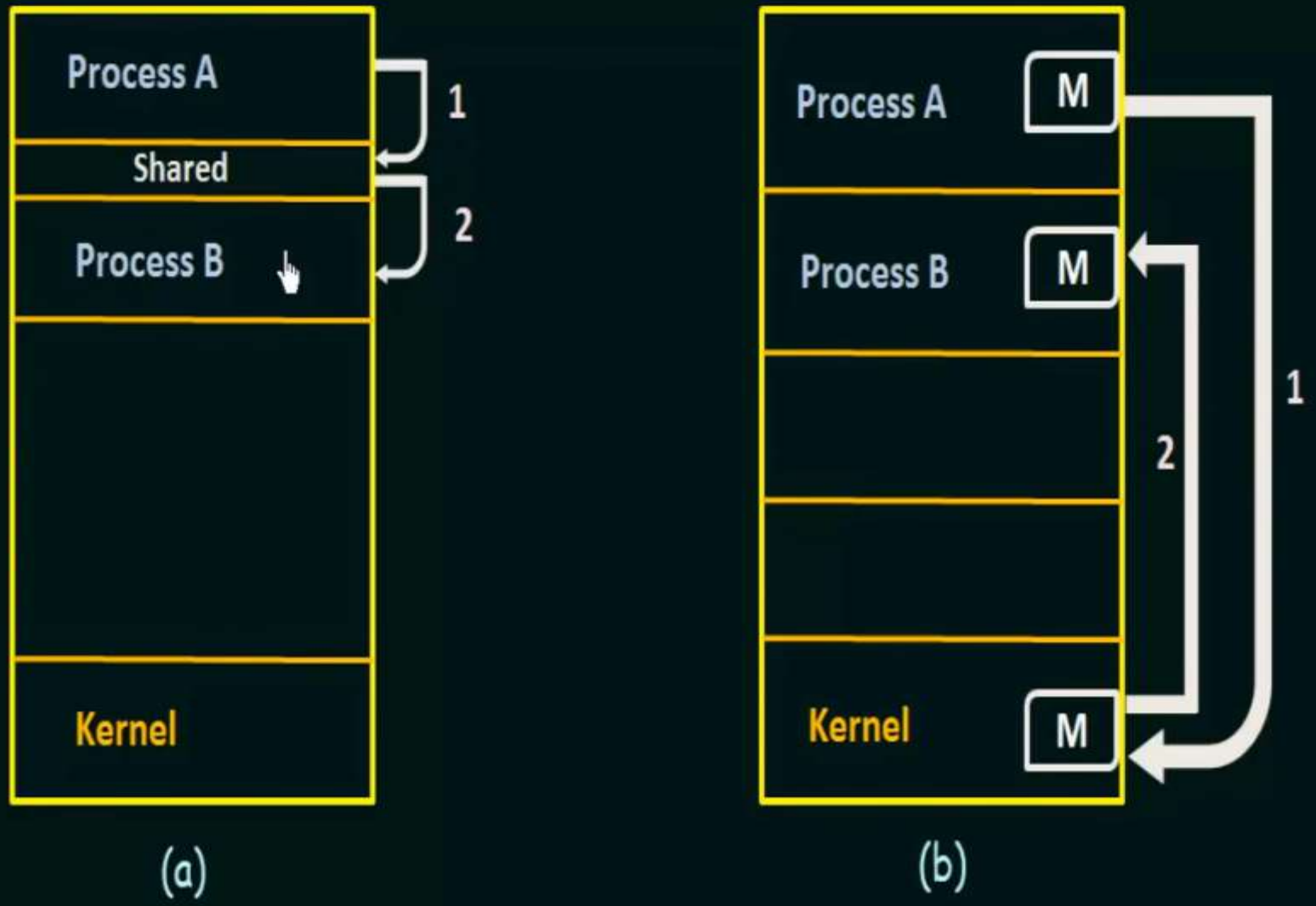
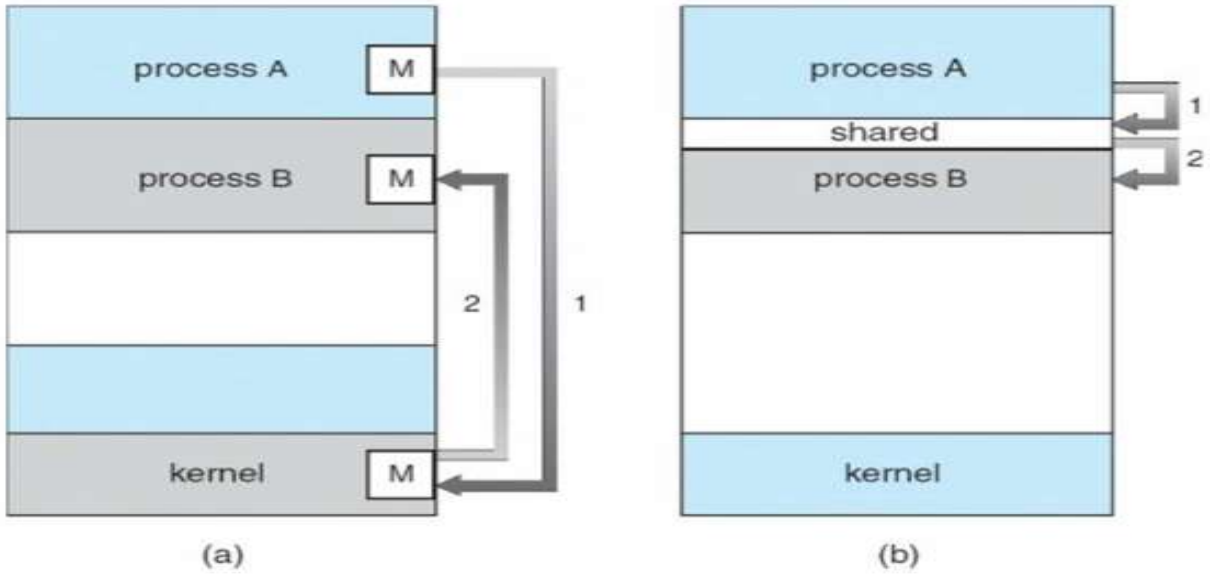


Fig: Communications models, (a) Shared memory, (b) Message Passing.

There are two fundamental ways of IPC

a. Message Passing b. Shared Memory



Shared Memory:

- ✓ Here a region of memory that is shared by co-operating process is established.
- ✓ Process can exchange the information by reading and writing data to the **shared region.**
- ✓ Shared memory allows maximum speed and convenience of communication as it can be done at the speed of memory within the computer.
- ✓ System calls are required only to establish shared memory regions.
- ✓ Once shared memory is established no assistance from the kernel is required, all access are treated as routine memory access

Message Passing:

- ✓ Communication takes place by means of messages exchanged between the co-operating process
- ✓ Message passing is useful for exchanging the smaller amount of data.
- ✓ Easier to implement than shared memory.
- ✓ Slower than that of Shared memory as message passing system are typically implemented using system call
- ✓ Which requires more time consuming task of Kernel intervention

Race Condition

- The situation where two or more processes are reading or writing some shared data, but not in proper sequence is called race Condition
- When two or more processes are executing the same code or resources or any stored variables in that condition there is a possibility that the output or value of that shared variables is wrong, so for that the all the process doing race to say that is correct , this condition is known as Race condition.

Critical Section:

- It is the part of program where shared resources are accessed by various process.
- The part of the program where the shared memory is accessed that must not be concurrently accessed by more then one processes is called the critical region.

❖ Four conditions to provide mutual exclusion Or (Rules for avoiding Race Condition) Solution to Critical section problem:

- ✓ No two processes simultaneously in critical region
- ✓ No assumptions made about speeds or numbers of CPUs
- ✓ No process running outside its critical region may block another process
- ✓ No process must wait forever to enter its critical region

Mutual Exclusion:

- Mutual exclusion implies that only one process can be inside the critical section at any time. If any other processes require the critical section, they must wait until it is free.
- A mutual exclusion (mutex) is a program object that prevents simultaneous access to a shared resource.
- This concept is used in concurrent programming with a critical section, a piece of code in which processes or threads access a shared resource.
- Only one thread owns the mutex at a time, thus a mutex with a unique name is created when a program starts.
- When a thread holds a resource, it has to lock the mutex from other threads to prevent concurrent access of the resource.
- Upon releasing the resource, the thread unlocks the mutex

Mutual Exclusion with Busy waiting

Busy waiting:

- ✓ Continuously testing a variable until some value appears is called busy waiting.
- ✓ If the entry is allowed it execute else it sits in tight loop and waits.
- ✓ ***Busy-waiting: consumption of CPU cycles while a thread is waiting for a lock***
  - ***Very inefficient***
  - Can be avoided with a waiting queue

Disabling Interrupts
(Hardware Solution)

- Each process disables all interrupts just after entering in its critical section and re-enable all interrupts just before leaving critical section.
- With interrupts turned off the CPU could not be switched to other process.
- Hence, no other process will enter its critical and mutual exclusion achieved.

Conclusion

- Disabling interrupts is sometimes a useful interrupts is sometimes a useful technique within the kernel of an operating system, but it is not appropriate as a general mutual exclusion mechanism for users process.
- The reason is that it is unwise to give user process the power to turn off interrupts

Lock Variable (Software Solution)

- In this solution, we consider a single, shared, (lock) variable, initially 0.
- When a process wants to enter in its critical section, it first test the lock.
- If lock is 0, the process first sets it to 1 and then enters the critical section.
- If the lock is already 1, the process just waits until (lock) variable becomes 0.
- Thus, a 0 means that no process in its critical section, and 1 means hold your horses - some process is in its critical section.

Conclusion

- The flaw in this proposal can be best explained by example.
- Suppose process A sees that the lock is 0. Before it can set the lock to 1 another process B is scheduled, runs, and sets the lock to 1.
- When the process A runs again, it will also set the lock to 1, and two processes will be in their critical section simultaneously.

Turn Variable or Strict Alternation Approach

- Turn Variable or Strict Alternation Approach is the software mechanism implemented at user mode.
- It is a busy waiting solution which can be implemented only for two processes.
- In this approach, A turn variable is used which is actually a lock.

Strict Alternation

Process 0	Process 1

While (TRUE) {
while (turn != 0); // wait
critical_section();
turn = 1;
noncritical_section();}
}
	While (TRUE) {
	while (turn != 1); // wait
	critical_section();
	turn = 0;
	noncritical_section();
	}

Assume the variable *turn* is initially set to zero.

- ✓ Process 0 is allowed to run.
- ✓ It finds that *turn* is zero and is allowed to enter its critical region.
- ✓ If process 1 tries to run, it will also find that *turn* is zero and will have to wait (the while statement) until *turn* becomes equal to 1.
- ✓ When process 0 exits its critical region it sets *turn* to 1, which allows process 1 to enter its critical region.
- ✓ If process 0 tries to enter its critical region again it will be blocked as *turn* is no longer zero.

However, there is one major flaw in this approach. Consider this sequence of events

- Process 0 runs, enters its critical section and exits; setting turn to 1. Process 0 is now in its non-critical section. Assume this non-critical procedure takes a long time.
- Process 1, which is a much faster process, now runs and once it has left its critical section turn is set to zero.
- Process 1 executes its non-critical section very quickly and returns to the top of the procedure.
- The situation is now that process 0 is in its non-critical section and process 1 is waiting for turn to be set to zero. In fact, there is no reason why process 1 cannot enter its critical region as process 0 is not in its critical region.

- What we can see here is violation of one of the conditions that is mutual exclusion.
- That is, a process, not in its critical section, is blocking another process.

- If you work through a few iterations of this solution you will see that the processes must enter their critical sections in turn; thus this solution is called ***strict alternation***

Paterson Solution

- This is a software mechanism implemented at user mode.
- It is a busy waiting solution can be implemented for only two processes.
- It uses two variables that are turn variable and interested variable.
- By combination the idea of taking turns with the idea of lock variables and warning variables.:

```
while (1)
{
                                    //repeat forever OR
while(TRUE) flag[0] =T; // interested to
 enter c.s turn=1;

while(turn==1 && flag[1]==T) /* loop to give chance to
other*/;
 critical_region();
flag[0] =F;
noncritical_region();
 }
```

(a) Process 0.

```
while (1)
{
//repeat forever

flag[1] =T; // interested to
enter c.s turn=0;
while(turn==0 && flag[0]==T) /* loop to give chance to other*/; critical_region();

flag[1] =F;
noncritical_region();
}
```

(b) Process 1.

The Code of the solution is given below

```
# define N 2
# define TRUE 1
# define FALSE 0
int interested[N] = FALSE;
int turn;
voidEntry_Section (int process)
{
   int other;
  other = 1-process;
 interested[process] = TRUE;
turn = process;
while (interested [other] =True && TURN=process);
}
voidExit_Section (int process)
{
  interested [process] = FALSE;
}
```

Analysis of Peterson Solution

```
voidEntry_Section (int process)
{
    1. int other;
    2. other = 1-process;
    3. interested[process] = TRUE;
    4. turn = process;
    5. while (interested [other] =True && TURN=process);
}

Critical Section

voidExit_Section (int process)
{
    6. interested [process] = FALSE;
}
```

- ✓ This is a two process solution.
- ✓ Let us consider two cooperative processes P1 and P2.
- ✓ The entry section and exit section are shown below.
- ✓ Initially, the value of interested variables and turn variable is 0.
- ✓ Initially process P1 arrives and wants to enter into the critical section.
- ✓ It sets its interested variable to True (instruction line 3) and also sets turn to 1 (line number 4).
- ✓ Since the condition given in line number 5 is completely satisfied by P1 therefore it will enter in the critical section.
- ✓ P1 → 1 2 3 4 5 CS

- Meanwhile, Process P1 got preempted and process P2 got scheduled.
- P2 also wants to enter in the critical section and executes instructions 1, 2, 3 and 4 of entry section.
- On instruction 5, it got stuck since it doesn't satisfy the condition (value of other interested variable is still true).
- Therefore it gets into the busy waiting.
- P2 → 1 2 3 4 5

- ❖P1 again got scheduled and finish the critical section by executing the instruction no. 6 (setting interested variable to false).
- ❖Now if P2 checks then it are going to satisfy the condition since other process's interested variable becomes false.
- ❖ P2 will also get enter the critical section.
- P1 → 6
- P2 → 5 CS
- ❖Any of the process may enter in the critical section for multiple numbers of times.
- ❖ Hence the procedure occurs in the cyclic order.

Mutual Exclusion:

- ✓ The method provides mutual exclusion for sure.
- ✓ In entry section, the while condition involves the criteria for two variables therefore a process cannot enter in the critical section until the other process is interested and the process is the last one to update turn variable.

Progress:

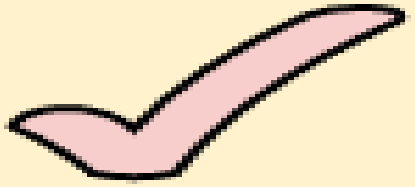
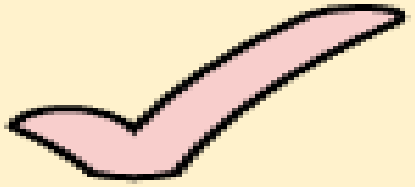
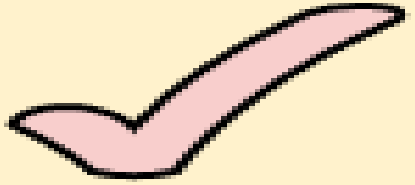
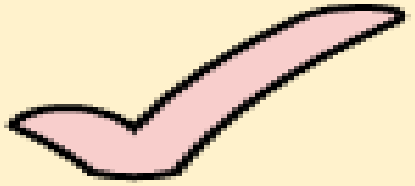
- ✓ An uninterested process will never stop the other interested process from entering in the critical section.
- ✓ If the other process is also interested then the process will wait

Bounded waiting:

- ✓ The interested variable mechanism failed because it was not providing bounded waiting.
- ✓ However, in Peterson solution, A deadlock can never happen because the process which first sets the turn variable will enter in the critical section for sure.
- ✓ Therefore, if a process is preempted after executing line number 4 of the entry section then it will definitely get into the critical section in its next chance.

Portability:

- This is the complete software solution and therefore it is portable on every hardware.

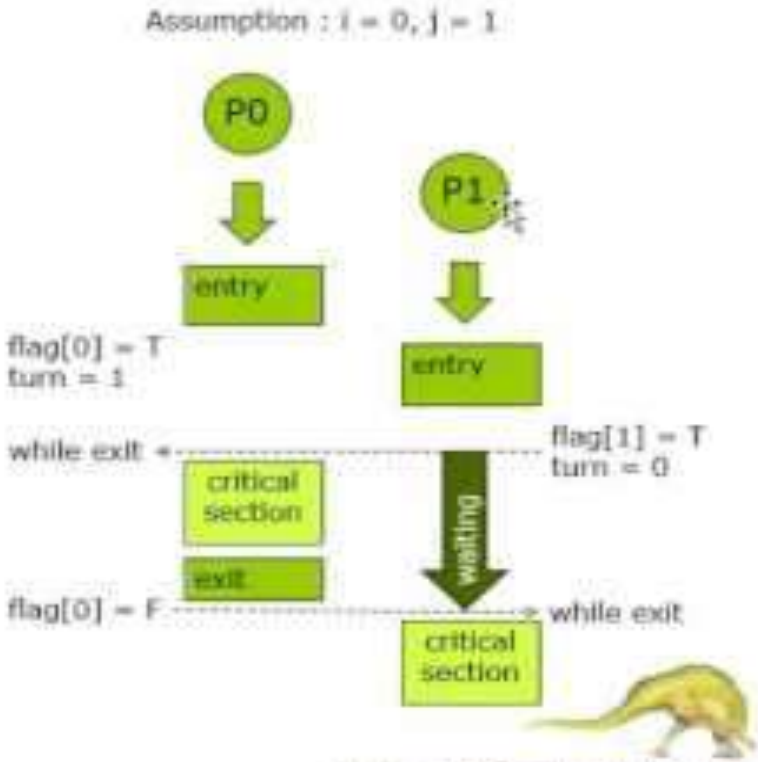
Mutual Exclusion	
Progress	
Bounded Waiting	
Portability	

3. Peterson's Solution

Algorithm for Process P_i

```
do {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j);
          critical section
    flag[i] = false;
          remainder section
} while (true);
```

- Provable that
1. Mutual exclusion is preserved
2. Progress requirement is satisfied
3. Bounded-waiting requirement is met



The TSL (Test and Set Lock)Instruction

- ✓ Many computers, especially those designed with multiple processors in mind, have an instruction:
- ✓ TSL RX,LOCK
- ❖ (Test and Set Lock) that works as follows.
- ✓ It reads the contents of the memory word *lock* into register RX and then stores a nonzero value at the memory address *lock.*
- ✓ The operations of reading the word and storing into it are guaranteed to be indivisible—no other processor can access the memory word until the instruction is finished.
- ✓ The CPU executing the TSL instruction locks the memory bus to prohibit other CPUs from accessing memory until it is done.

- ✓When *lock* is 0, any process may set it to 1 using the TSL instruction and then read or write the shared memory.
- ✓ When it is done, the process sets *lock* back to 0 using an ordinary move instruction.

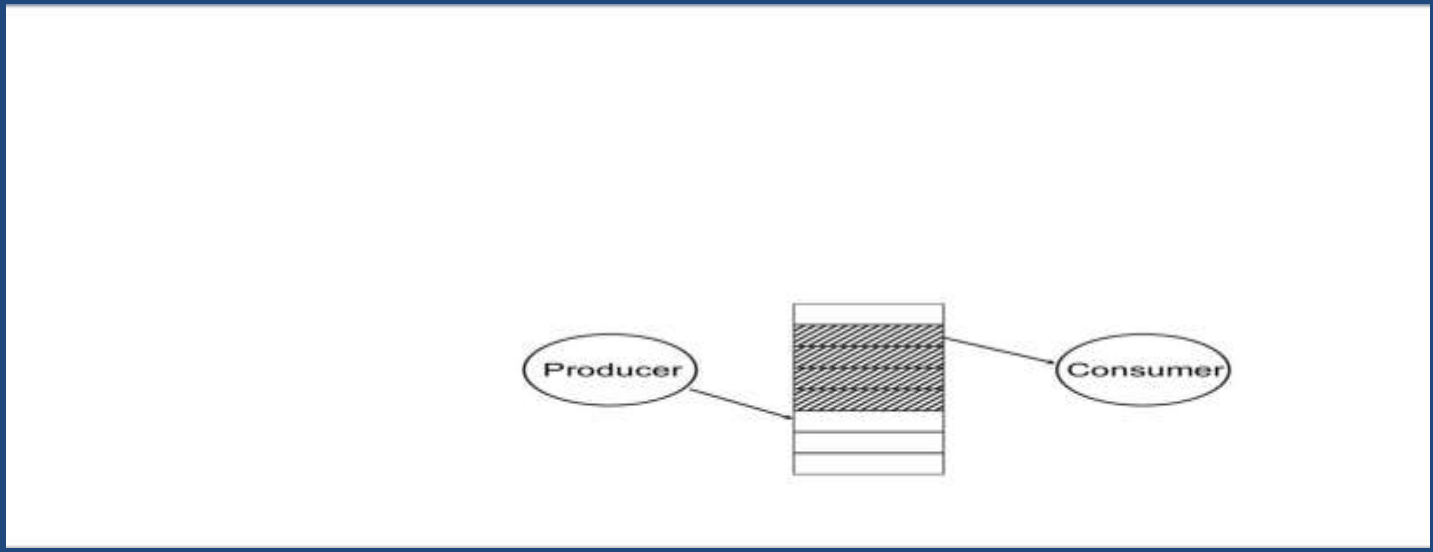
Sleep and Wakeup:

- ✓ Sleep and wakeup are system calls that blocks process instead of wasting CPU time when they are not allowed to enter their critical region.
- ✓ Sleep is a system call that causes the caller to block, that is, be suspended until another process wakes it up.
- ✓ The wakeup call has one parameter, the process to be awakened eg. wakeup(consumer) or wakeup(producer).

- ✓ When a process wants to enter its critical region, it checks to see if the entry is allowed. If it is not allowed, the process just sits in a tight loop waiting until it is allowed.
- ✓ Beside of wasting CPU time, this approach can also have unexpected effects.
- ✓ Consider a computer with two processes, H, with high priority and L, with low priority.
- ✓ The scheduling rules are such that H runs whenever it is in ready state. At a certain moment, with L is in its critical region, H becomes ready to run. H now begins busy waiting.
- ✓ Before H is completed, L can not be scheduled. So L never gets the chance to leave its critical region, so H loops forever. 84

Producer-consumer problem (Bounded Buffer):

- ✓Two processes share a common, fixed-size buffer.
- ✓One of them, the producer, puts information into the buffer, and the other one, the consumer, takes it out



Trouble arises when

1. The producer wants to put a new data in the buffer, but buffer is already full.

Solution:

Producer goes to sleep and to be awakened when the consumer has removed data.

2. The consumer wants to remove data from the buffer but buffer is already empty.

Solution:

Consumer goes to sleep until the producer puts some data in buffer and wakes consumer up.

```
#define N 100 /* number of slots in the buffer */
int count = 0; /* number of items in the buffer */
void producer(void)
{
int item;
while (TRUE)
{ /* repeat forever */
item = produce_item(); /* generate next item */
if (count == N) sleep(); /* if buffer is full, go to sleep */
insert_item(item); /* put item in buffer */
count = count + 1; /* increment count of items in buffer */
if (count == 1) wakeup(consumer); /* was buffer empty?
ie. initially */
}
}
```

```
void consumer(void)
{
 int item;
while (TRUE)
{ /* repeat forever */
if (count == 0) sleep(); /* if buffer is empty, got to sleep */
item = remove_item(); /* take item out of buffer */
count = count - 1; /* decrement count of items in buffer */
if (count ==N - 1) wakeup(producer); /* was buffer full? */
consume_item(item); /*consume item */
}
}
```

Count--> a variable to keep track of the no. of items in the buffer. N → Size of Buffer

Producers code:

- ✓ The producers code is first test to see if count is N.
- ✓ If it is, the producer will go to sleep ; if it is not the producer will add an item and increment count.

Consumer code:

- ✓ It is similar as of producer.
- ✓ First test count to see if it is 0. If it is, go to sleep; if it nonzero remove an item and decrement the counter.
- ✓ Each of the process also tests to see if the other should be awakened and if so wakes it up.
- ✓ This approach sounds simple enough, but it leads to the same kinds of **race conditions as we saw in the spooler directory.**

- **Lets see how the race condition arises**

✓ The buffer is empty and the consumer has just read count to see if it is 0.

✓ At that instant, the scheduler decides to stop running the consumer temporarily and start running the producer. (Consumer is interrupted and producer resumed)

✓ The producer creates an item, puts it into the buffer, and increases count.

✓ Because the buffer was empty prior to the last addition (count was just 0), the producer tries to wake up the consumer.

- Unfortunately, the consumer is not yet logically asleep, so the **wakeup signal is lost.**
- When the consumer next runs, it will test the value of count it previously read, find it to be 0, and go to sleep.
- Sooner or later the producer will fill up the buffer and also go to sleep. Both will sleep forever.

✓ The problem here is that a wakeup sent to a process that is not(yet) sleeping is lost.

Semaphores

- Semaphore is an integer variable to count the number of wakeup processes saved for future use.
- A semaphore could have the value 0, indicating that no wakeup processes were saved, or some positive value if one or more wakeups were pending.
- There are two operations, down (wait) and up(signal) (generalizations of sleep and wakeup, respectively).
- The down operation on a semaphore checks if the value is greater than 0. If so, it decrements the value and just continues.
- If the value is 0, the process is put to sleep without completing the down for the moment.

- The up operation increments the value of the semaphore addressed.
- If one or more processes were sleeping on that semaphore, unable to complete an earlier down operation, one of them is chosen by the system (e.g., at random) and is allowed to complete its down.
- According to the demand of the situation, Semaphore can be divided into two categories.

- Counting Semaphore
- Binary Semaphore or Mutex

❖Counting semaphore:

- ✓integer value can range over an unrestricted domain

❖Binary semaphore:

- ✓integer value can range only between 0 and 1

can be simpler to implement Also known as mutex locks

Semaphore operations:

- P or Down, or Wait: **P stands for *proberen ("to test")***
- V or Up or Signal: Dutch words. **V stands for *verhogen ("increase")***
- **wait(sem) :**
- ✓ decrement the semaphore value. if negative, suspend the process and place in queue. (Also referred to as *P(), down in literature.)*
- **signal(sem)**
- ✓ increment the semaphore value, allow the first process in the queue to continue. (Also referred to as *V(), up in literature.)*

Implementation of wait:

- wait(s)

```
{
while(s<=0); // do nothing
s=s-1;
}
```

Implementation of signal:

```
signal(s)
{
 s=s+1;
}
```

Advantages of semaphores

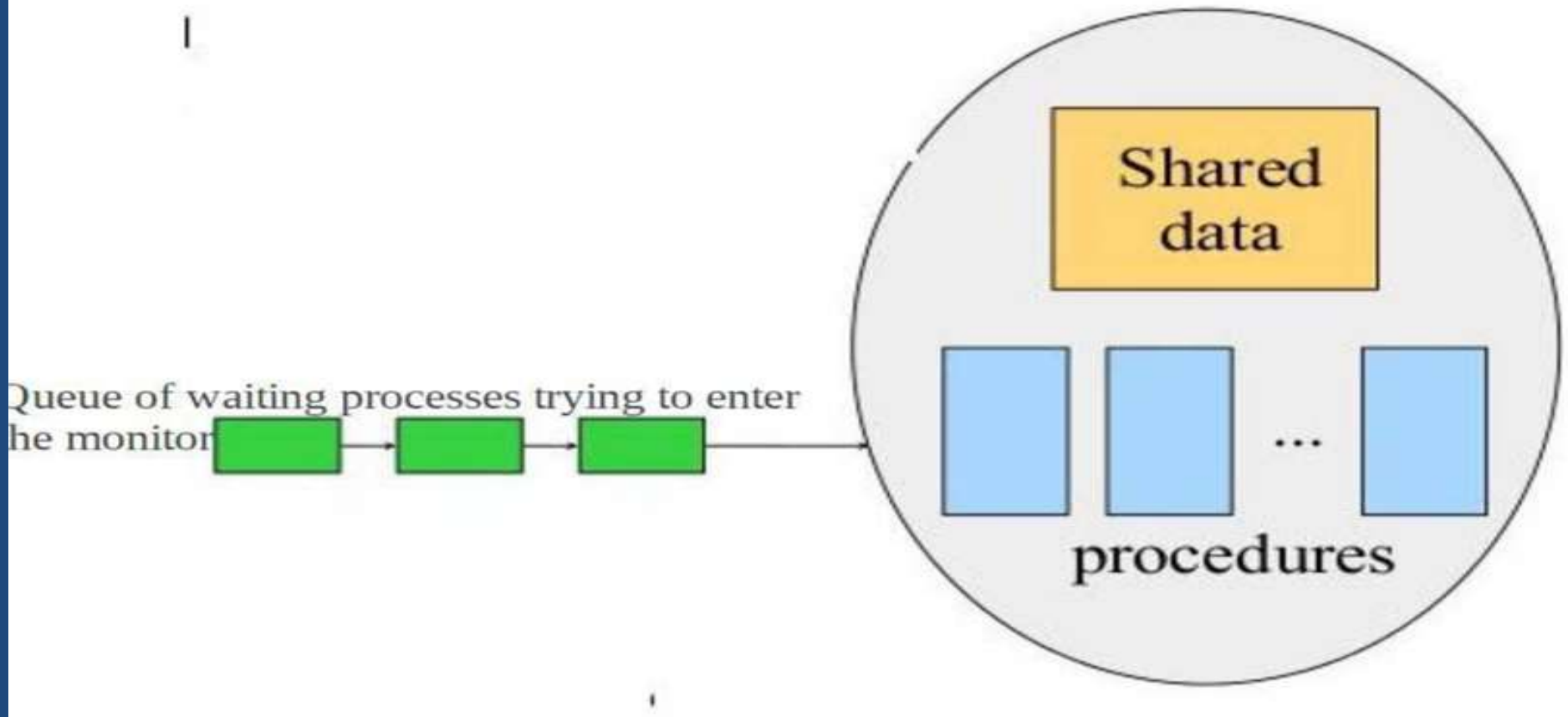
- ✓ Processes do not busy wait while waiting for resources.
- ✓ While waiting, they are in a "suspended" state, allowing the CPU to perform other work.
- ✓ Works on (shared memory) multiprocessor systems.
- ✓ User controls synchronization.

Disadvantage of semaphores

- ✓ can only be invoked by processes--not interrupt service routines because interrupt routines cannot block
- ✓ user controls synchronization--could mess up.

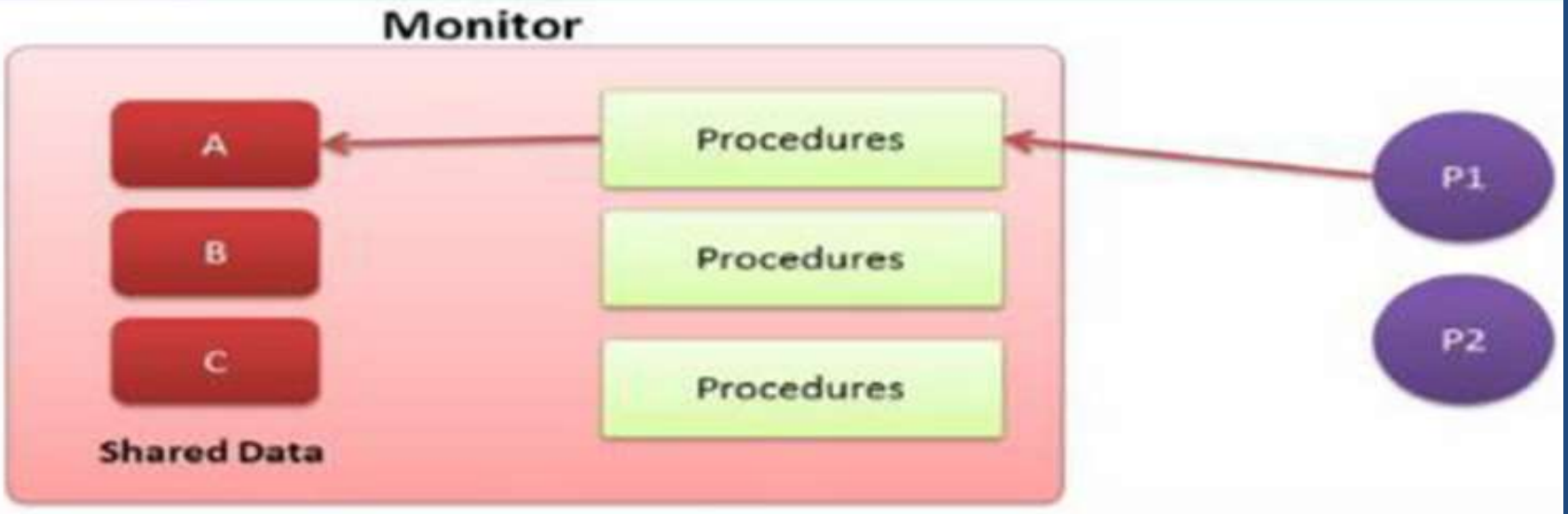
Monitors:

- ✓ In concurrent programming, a **monitor is an object or module** intended to be used safely by more than one thread.
- ✓ The defining characteristic of a monitor is that its methods are executed with mutual exclusion.
- ✓ That is, at each point in time, at most one thread may be executing any of its methods.
- ✓ Monitors also provide a mechanism for threads to temporarily give up exclusive access, it has to wait for some condition to be met, before regaining exclusive access and resuming their task.
- ✓ Monitors also have a mechanism for signaling other threads that such conditions have been met.


Queue of waiting processes trying to enter
he monitor
Shared
data
...
procedures

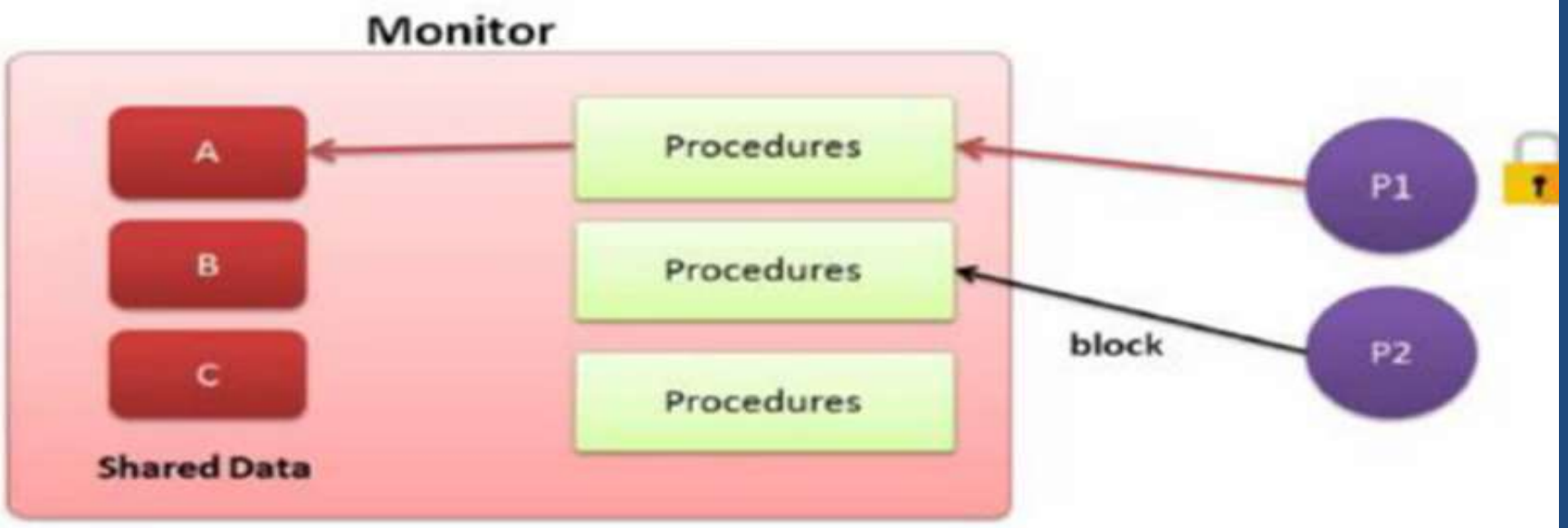
- ✓ A monitor is a collection of procedures, variables, and data structures that are all grouped together in a special kind of module or package.
- ✓ Processes may call the procedures in a monitor whenever they want to, but they cannot directly access the monitor's internal data structures from procedures declared outside the monitor

Monitor



A monitor is a module that encapsulates
- Shared data structures
- Procedures that operates on the shared data
- Synchronization between concurrent procedure invocation

Monitor



Only one thread can enter in monitor at any time.

Message Passing

- ✓ Interprocess communication uses two primitives, send and receive.
- ✓ They can easily be put into library procedures, such as *send(destination, &message);*

receive(source, &message);

- ✓ The former call sends a message to a given destination and the latter one receives a message from a given source.
- ✓ If no message is available, the receiver can block until one arrives. Alternatively, it can return immediately with an error code

2.5 classical IPC problem:

- Producer Consumer Problem
- Sleeping Barber Problem
- Dining Philosopher Problem

Producer Consumer Problem

- This solution uses three semaphores:
  - ✓ one called *full* for counting the number of slots that are full,
  - ✓ one called *empty* for counting the number of slots that are empty, and
  - ✓ one called *mutex* to make sure the producer and consumer do not access the buffer at the same time.
- *Full* is initially 0, *empty* is initially equal to the number of slots in the buffer (N), and *mutex* is initially 1.
- The *mutex* semaphore is used for mutual exclusion.
- It is designed to guarantee that only one process at a time will be reading or writing the buffer and the associated variables.
- The other use of semaphores is for **synchronization**.
- The *full* and *empty* semaphores are used to guarantee synchronization.
- In this case, they ensure that the producer stops running when the buffer is full, and the consumer stops running when it is empty.

```
#define N 100                    /* number of slots in the buffer */

typedef int semaphore;           /* semaphores are a special kind of int */
semaphore mutex = 1;             /* controls access to critical region */
semaphore empty = N;             /* counts empty buffer slots */
semaphore full = 0;              /* counts full buffer slots */

void producer(void)

  int item;

  while (TRUE) {                 /* TRUE is the constant 1 */
      item = produce_item();     /* generate something to put in buffer */
      down(&empty);              /* decrement empty count */
      down(&mutex);              /* enter critical region */
      insert_item(item);         /* put new item in buffer */
      up(&mutex);                /* leave critical region */
      up(&full);                 /* increment count of full slots */
  }


void consumer(void)

  int item;

  while (TRUE) {                 /* infinite loop */
      down(&full);               /* decrement full count */
      down(&mutex);              /* enter critical region */
      item a= remove_item();     /* take item from buffer */
      up(&mutex);                /* leave critical region */
      up(&empty);                /* increment count of empty slots */
      consume_item(item);        /* do something with the item */
```

Sleeping Barber problem

- ✓ There is a barber shop which has one barber, one barber chair, and n chairs for waiting for customers if there are any to sit on the chair.
- ✓ If there is no customer, then the barber sleeps in his own chair.
- ✓ When a customer arrives, he has to wake up the barber.
- ✓ If there are many customers and the barber is cutting a customer's hair, then the remaining customers either wait if there are empty chairs in the waiting room or they leave if no chairs are empty.


Waiting Chairs
Customer is leaving
Sleeping Barber
Customer is enterin

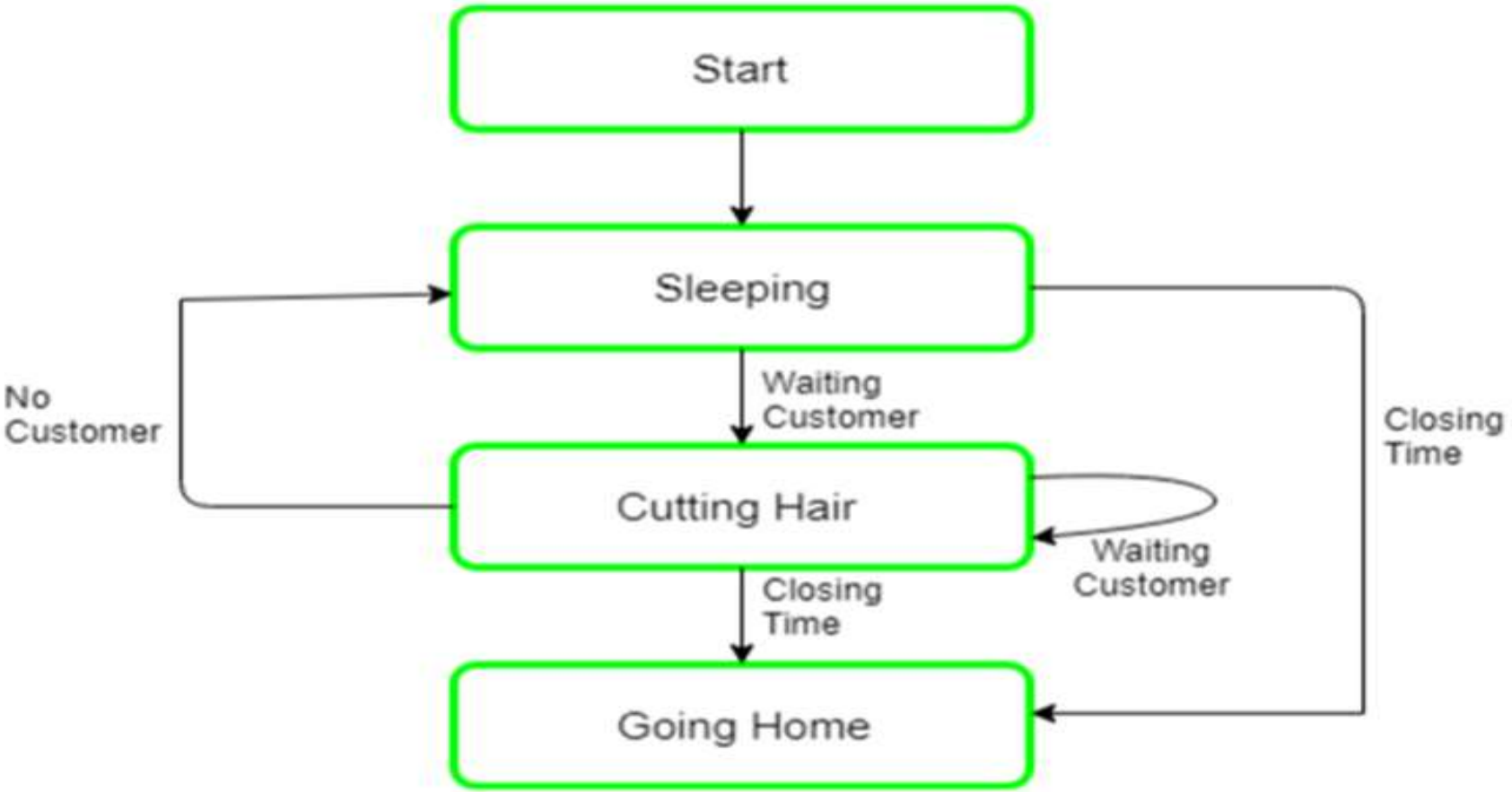
Solution :

The solution to this problem includes three semaphores.

- First is for the customer which counts the number of customers present in the waiting room (customer in the barber chair is not included because he is not waiting).
- Second, the barber 0 or 1 is used to tell whether the barber is idle or is working,
- And the third mutex is used to provide the mutual exclusion which is required for the process to execute.
- In the solution, the customer has the record of the number of customers waiting in the waiting room if the number of customers is equal to the number of chairs in the waiting room then the upcoming customer leaves the barbershop.

- When the barber shows up in the morning, he executes the procedure barber, causing him to block on the semaphore customers because it is initially 0. Then the barber goes to sleep until the first customer comes up.
- When a customer arrives, he executes customer procedure the customer acquires the mutex for entering the critical region, if another customer enters thereafter, the second one will not be able to anything until the first one has released the mutex.
- The customer then checks the chairs in the waiting room if waiting customers are less then the number of chairs then he sits otherwise he leaves and releases the mutex.

- If the chair is available then customer sits in the waiting room and increments the variable waiting value and also increases the customer’s semaphore this wakes up the barber if he is sleeping.
- At this point, customer and barber are both awake and the barber is ready to give that person a haircut. When the haircut is over, the customer exits the procedure and if there are no customers in waiting room barber sleeps.


Start
Sleeping
Waiting
Customer
No
Customer
Closing
Time
Cutting Hair
Waiting
Customer
Closing
Time
Going Home

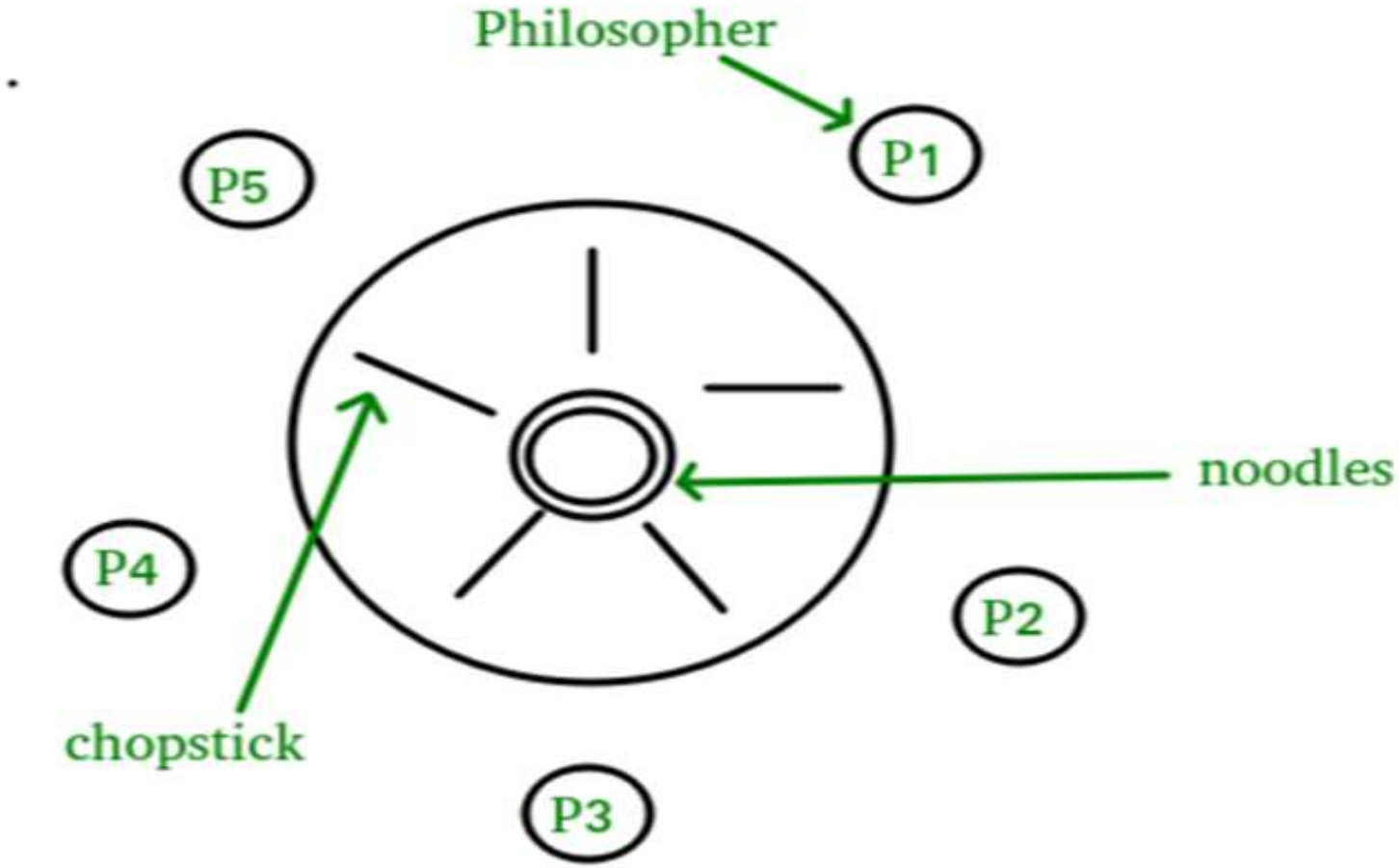
Algorithm for Sleeping Barber problem:

```
Semaphore Customers = 0;
Semaphore Barber = 0;
Mutex Seats = 1;
int FreeSeats = N;
Barber {
while(true) {        /* waits for a customer (sleeps). */
down(Customers);              /* mutex to protect the number of available seats.*/
 down(Seats);       /* a chair gets free.*/
FreeSeats++;        /* bring customer for haircut.*/
up(Barber);         /* release the mutex on the chair.*/
up(Seats);          /* barber is cutting hair.*/
}
}
```

```
Customer {
while(true) { /* protects seats so only 1 customer tries to sit in a chair if that's the case.*/
down(Seats); //This line should not be here.
if(FreeSeats > 0) {     /* sitting down.*/
FreeSeats--; /* notify the barber. */
 up(Customers); /* release the lock */
up(Seats);       /* wait in the waiting room if barber is busy. */
down(Barber);           // customer is having hair cut
} else {           /* release the lock */
 up(Seats);      // customer leaves
}
}
}
```

Dining Philosopher Problem

- The Dining Philosopher Problem states that N philosophers seated around a circular table with one chopstick between each pair of philosophers.
- There is one chopstick between each philosopher. A philosopher may eat if he can pickup the two chopsticks adjacent to him. One chopstick may be picked up by any one of its adjacent followers but not both.


Philosopher
P1
P5
noodles
P4
P2
chopstick
P3

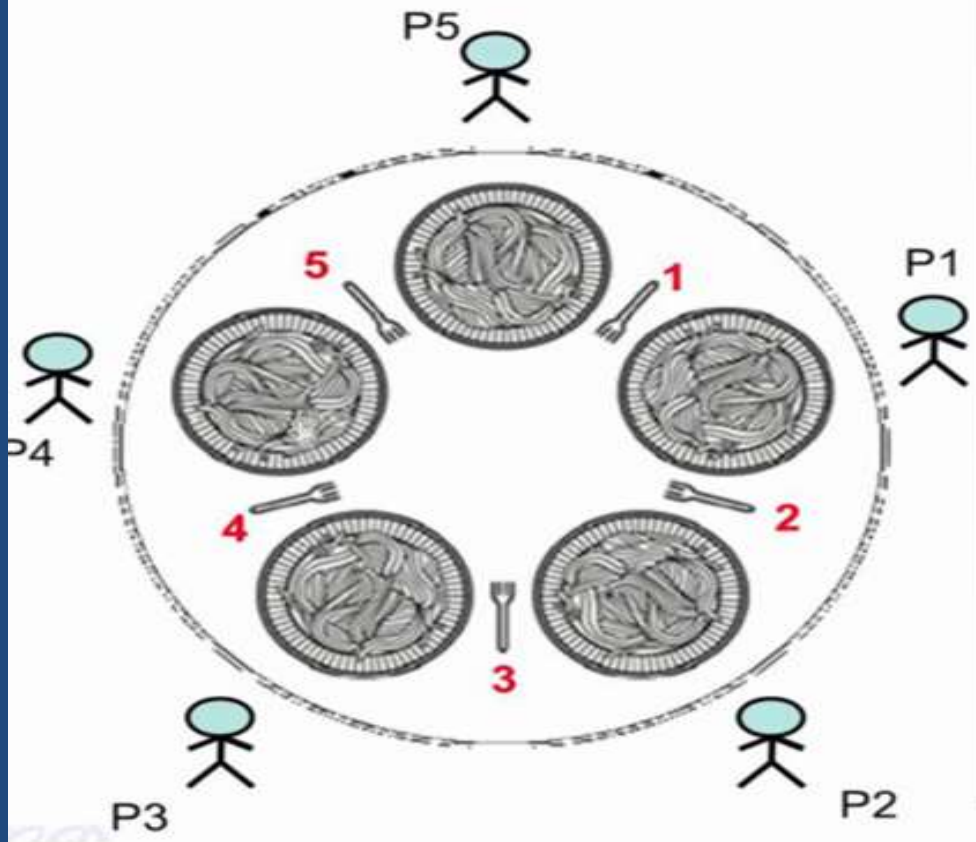
- We need an algorithm for allocating these limited resources(chopsticks) among several processes(philosophers) such that solution is free from deadlock and free from starvation.
- There exist some algorithm to solve Dining – Philosopher Problem, but they may have deadlock situation.
- Also, a deadlock-free solution is not necessarily starvation-free.
- Semaphores can result in deadlock due to programming errors.
- Monitors alone are not sufficiency to solve this, we need monitors with *condition variables*
-

Monitor-based Solution to Dining Philosophers

- ✓ We illustrate monitor concepts by presenting a deadlock-free solution to the dining-philosophers problem.
- ✓ Monitor is used to control access to state variables and condition variables.
- ✓ It only tells when to enter and exit the segment.
- ✓ This solution imposes the restriction that a philosopher may pick up her chopsticks only if both of them are available.

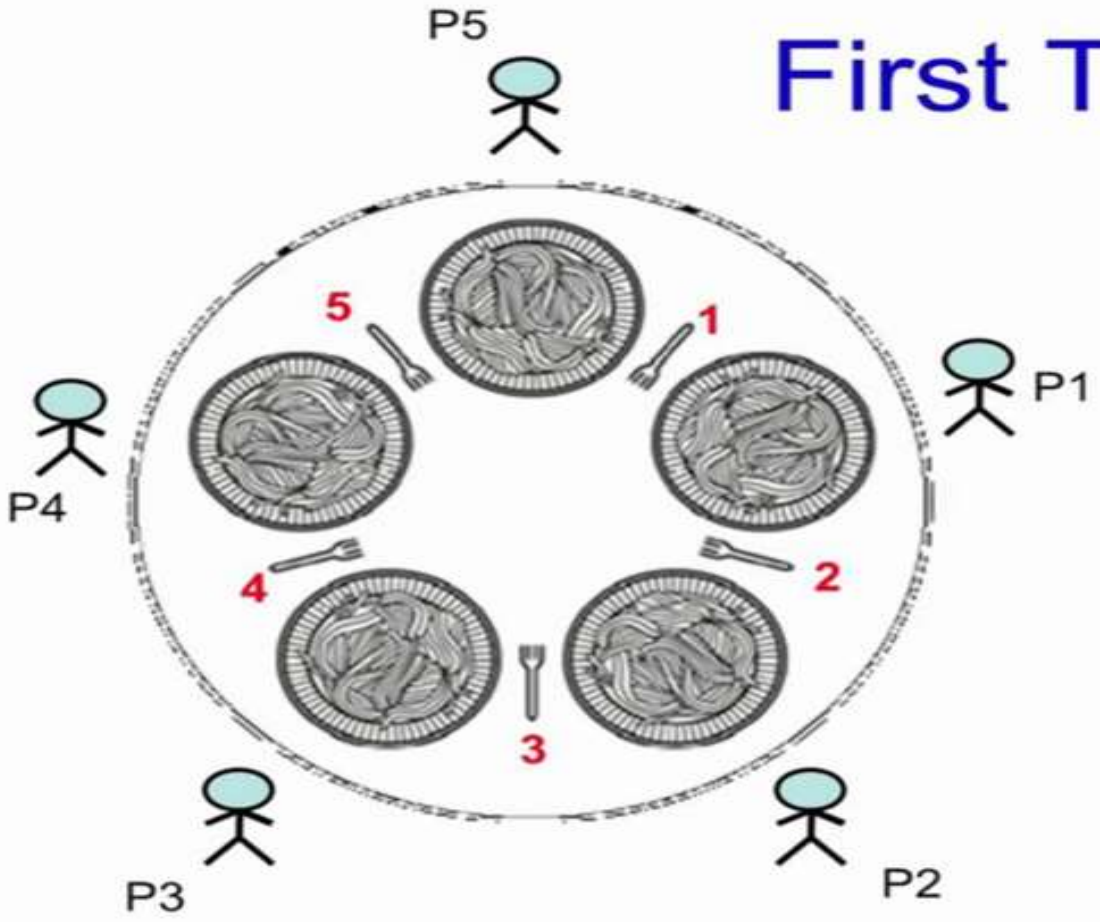
- ✓ To code this solution, we need to distinguish among three states in which we may find a philosopher.
- ✓ For this purpose, we introduce the following data structure:
  - **THINKING** – When philosopher doesn't want to gain access to either fork.
  - **HUNGRY** – When philosopher wants to enter the critical section.
  - **EATING** – When philosopher has got both the forks, i.e., he has entered the section.
- ✓ Philosopher i can set the variable state[i] = EATING only if
- ✓ her two neighbors are not eating (state[(i+4) % 5] != EATING) and (state[(i+1) % 5] != EATING).

Dining Philosophers Problem



- **Philosophers either think or eat**
- To eat, a philosopher needs to hold both forks (the one on his left and the one on his right)
- If the philosopher is not eating, he is thinking.
- Problem Statement : Develop an algorithm where no philosopher starves.

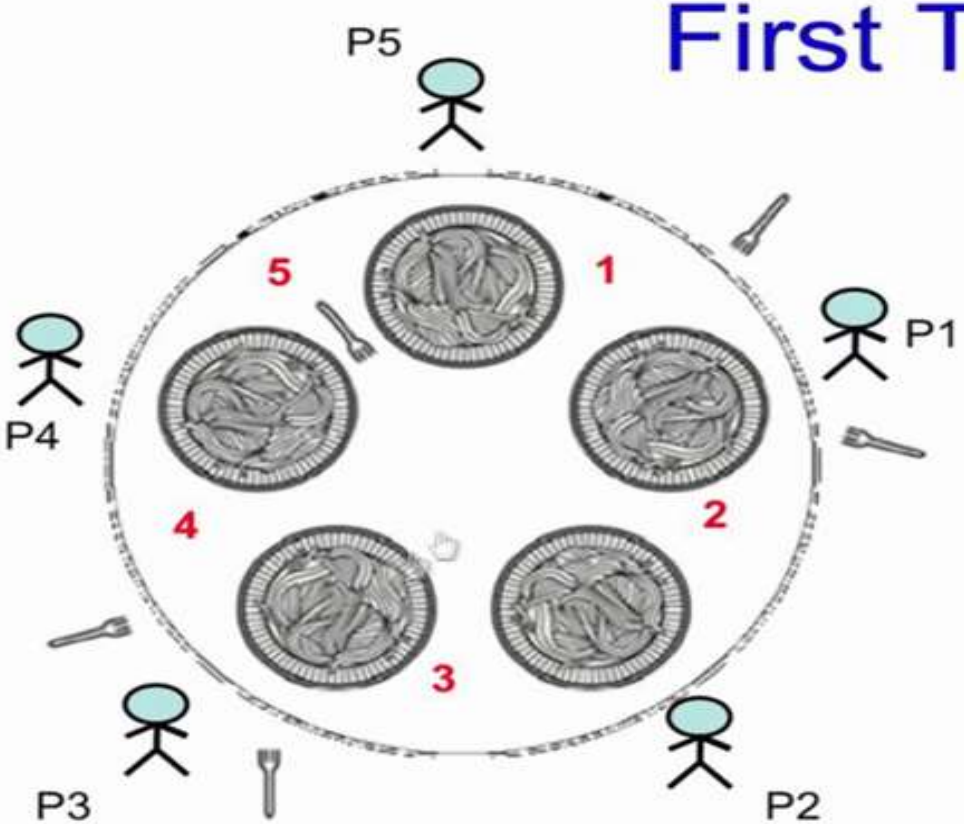
First Try




```
#define N 5

void philosopher(int i){
  while(TRUE){
      think(); // for some_time
      take_fork(Ri);
      take_fork(Li);
      eat();
      put_fork(Li);
      put_fork(Ri);
  }
}
```

First Try

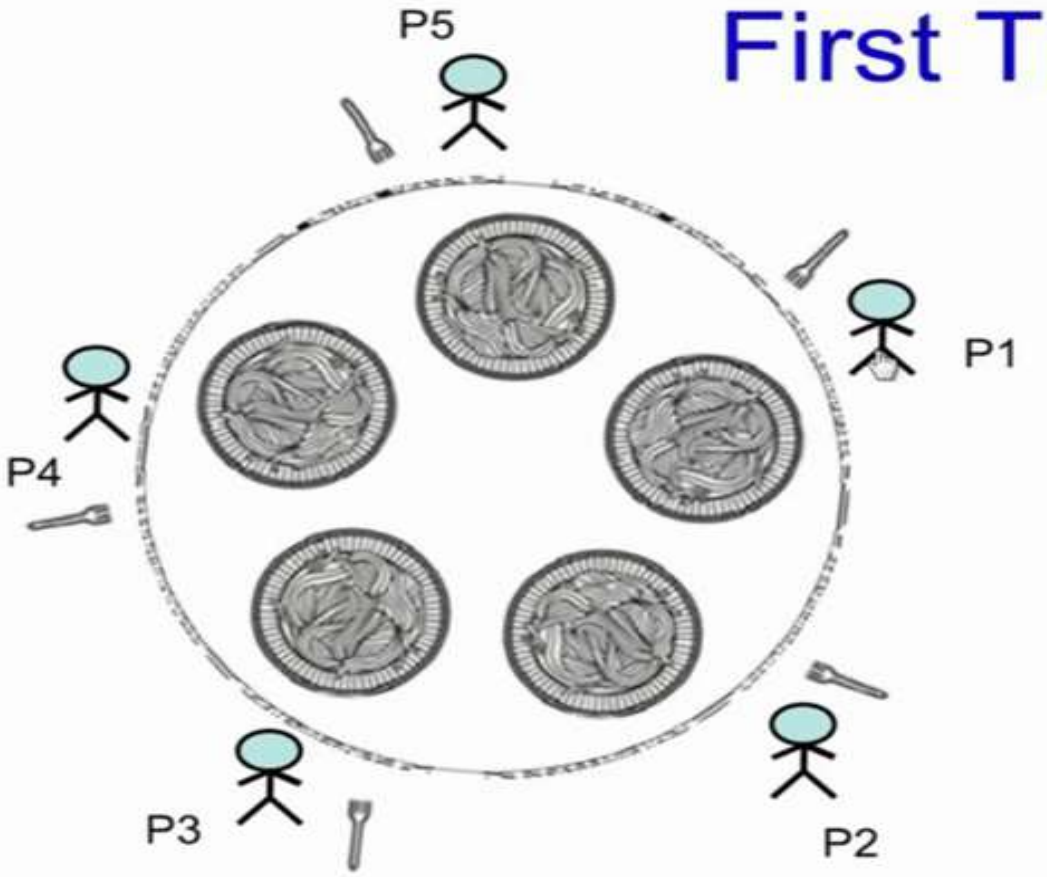



```
#define N 5

void philosopher(int i){
  while(TRUE){
      think(); // for some_time
      take_fork(Ri);
      take_fork(Li);
      eat();
      put_fork(Li);
      put_fork(Ri);
  }
}
```

What happens if only philosophers P1 and P3 are always given the priority?
P4, P5, and P2 starves… so scheme needs to be fair

First Try




```
#define N 5

void philosopher(int i){
 while(TRUE){
     think(); // for some_time
     take_fork(Ri);
     take_fork(Li);
     eat();
     put_fork(Li);
     put_fork(Ri);
 }
}
```

What happens if all philosophers decide to pick up their right forks at the same tim

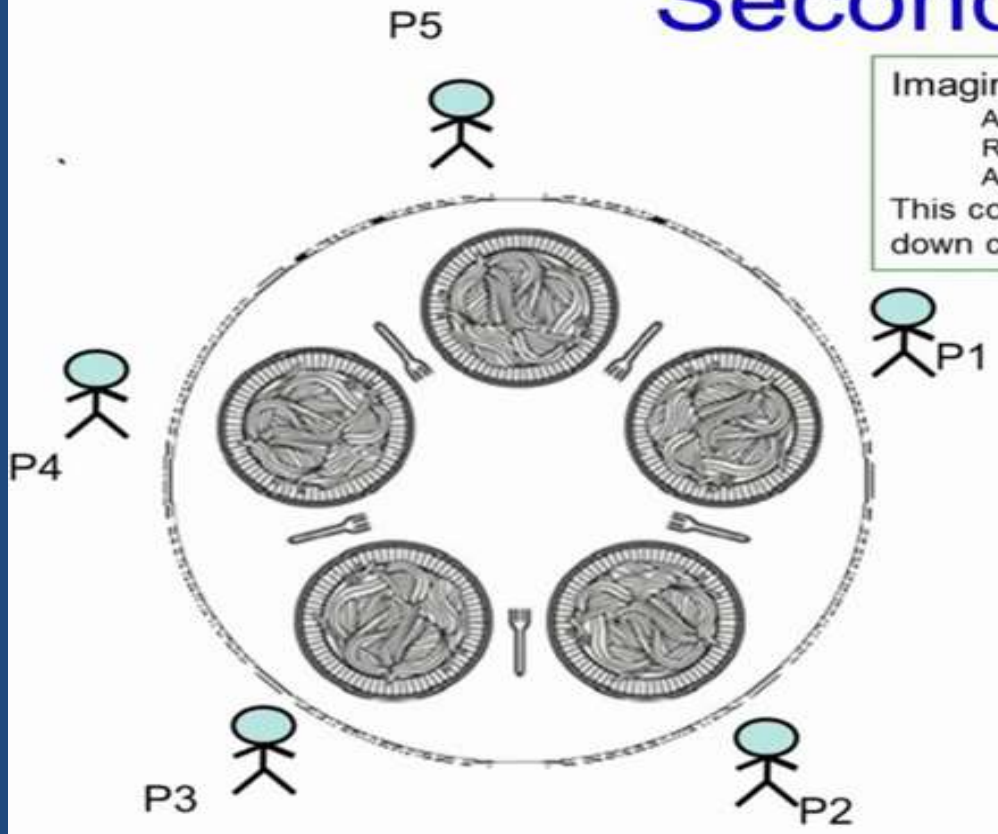
Possible starvation due to deadlock

Imagine,

All philosophers start at the same time
Run simultaneously
And think for the same time

This could lead to philosophers taking fork and putting it down continuously. a deadlock.

Second try

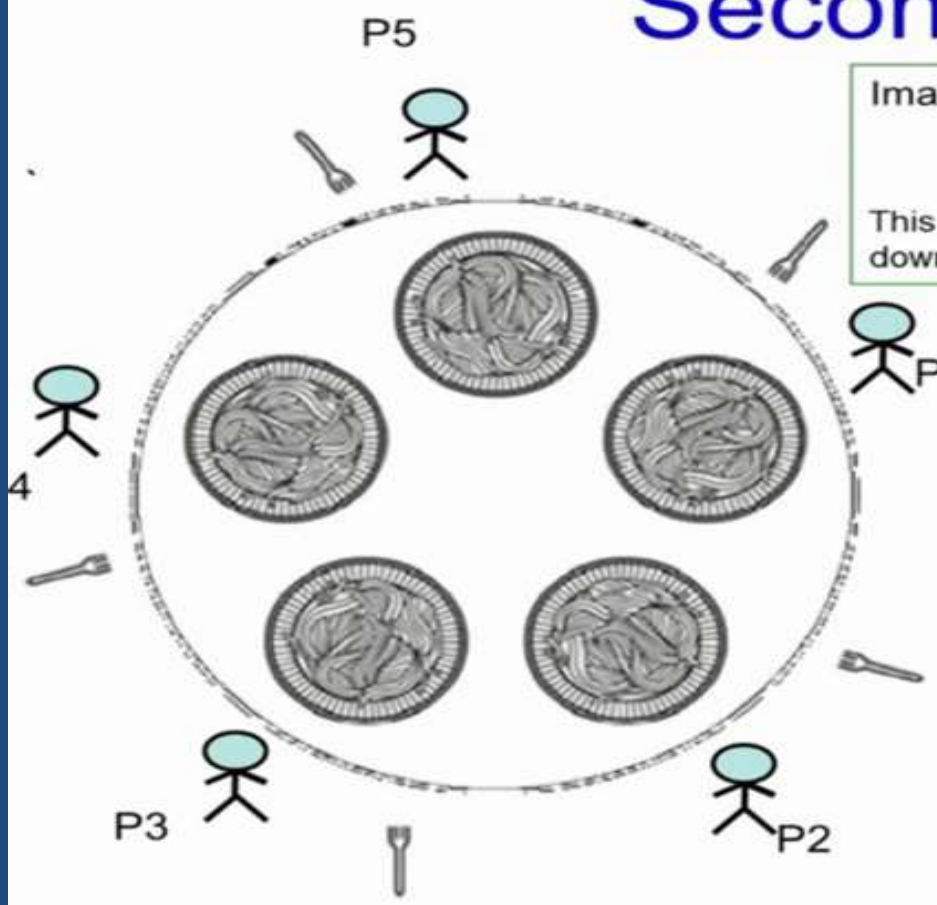


Imagine,
- All philosophers start at the same time
- Run simultaneously
- And think for the same time

This could lead to philosophers taking fork and putting it down continuously. a deadlock.

```
while(TRUE){
    think();
    take_fork(Ri);
    if (available(Li){
      take_fork(Li);
      eat();
      put_fork(Ri);
      put_fork(Li);
    }else{
       put_fork(Ri);
       sleep(T)
    }
}
```

Second try

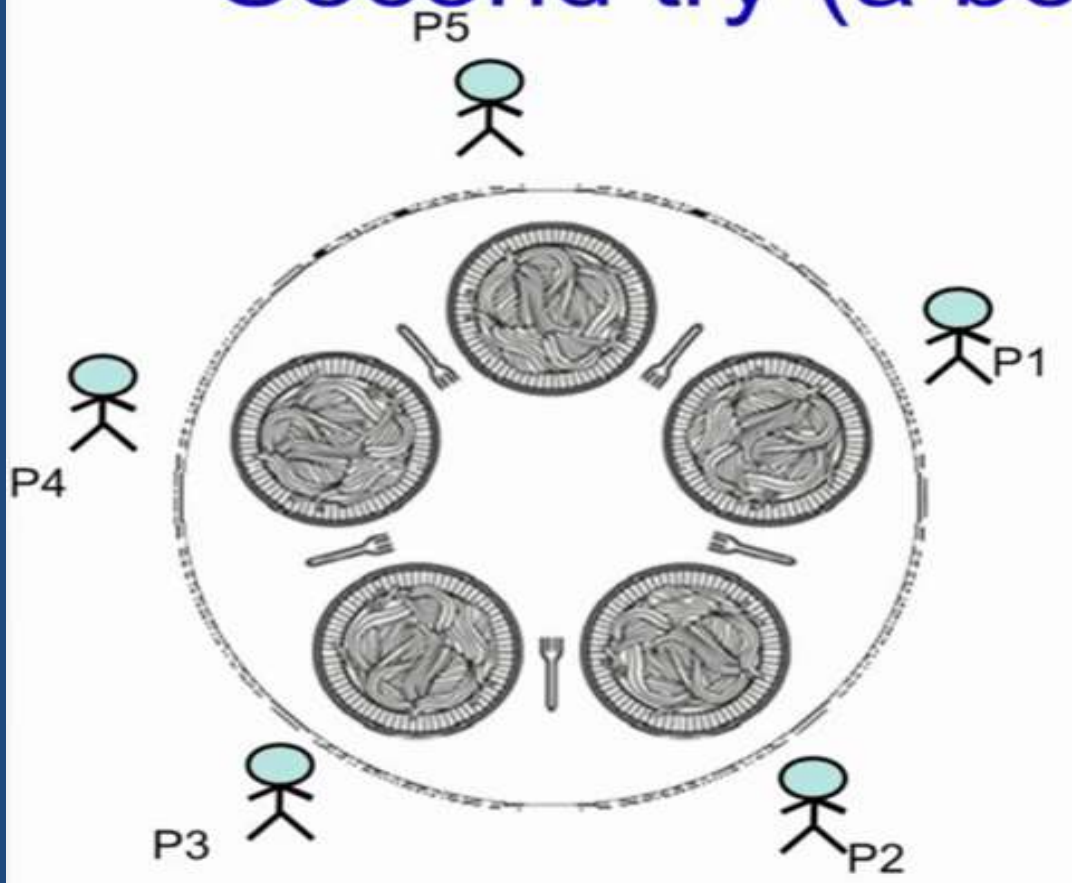


Imagine,
- All philosophers start at the same time
- Run simultaneously
- And think for the same time

This could lead to philosophers taking fork and putting it down continuously. a deadlock.

```
while(TRUE){
    think();
    take_fork(Ri);
    if (available(Li){
      take_fork(Li);
      eat();
      put_fork(Ri);
      put_fork(Li);
    }else{
       put_fork(Ri);
       sleep(T)
    }
}
```

Second try (a better solution)




```
#define N 5

void philosopher(int i){
  while(TRUE){
      think();
      take_fork(Ri);
      if (available(Li){
        take_fork(Li);
        eat();
        put_fork(Li);
        put_fork(Ri);
      }else{
         put_fork(Ri);
         sleep(random_time);
      }
}
```

Process Scheduling:

Definition:

- The process scheduling is the activity of the process manager that handles the removal of the running process from the CPU and the selection of another process on the basis of a particular strategy.
- Process scheduling is an essential part of Multiprogramming operating systems.
- Such operating systems allow more than one process to be loaded into the executable memory at a time and the loaded process shares the CPU using time multiplexing.

Scheduling

- In a multiprogramming system, frequently multiple process competes for the CPU at the same time.
- When two or more process are simultaneously in the ready state a choice has to be made which process is to run next.
- This part of the OS is called Scheduler and the algorithm is called scheduling algorithm

Scheduling Criteria:

Many criteria have been suggested for comparison of CPU scheduling algorithms.

CPU utilization:

- ✓ we have to keep the CPU as busy as possible.
- ✓ It may range from 0 to 100%.
- ✓ In a real system it should range from 40 – 90 % for lightly and heavily loaded system.

Throughput:

- ✓ It is the measure of work in terms of number of process completed per unit time.
- ✓ Eg: For long process this rate may be 1 process per hour, for short transaction, throughput may be 10 process per second

Turnaround Time:

- ✓ It is the sum of time periods spent in waiting to get into memory, waiting in ready queue, execution on the CPU and doing I/O.
- ✓ The interval form the time of submission of a process to the time of completion is the turnaround time.
- ✓ Waiting time plus the service time.
- ✓ **Turnaround time**= Time of completion of job - Time of submission of job. (waiting time + service time or burst time)

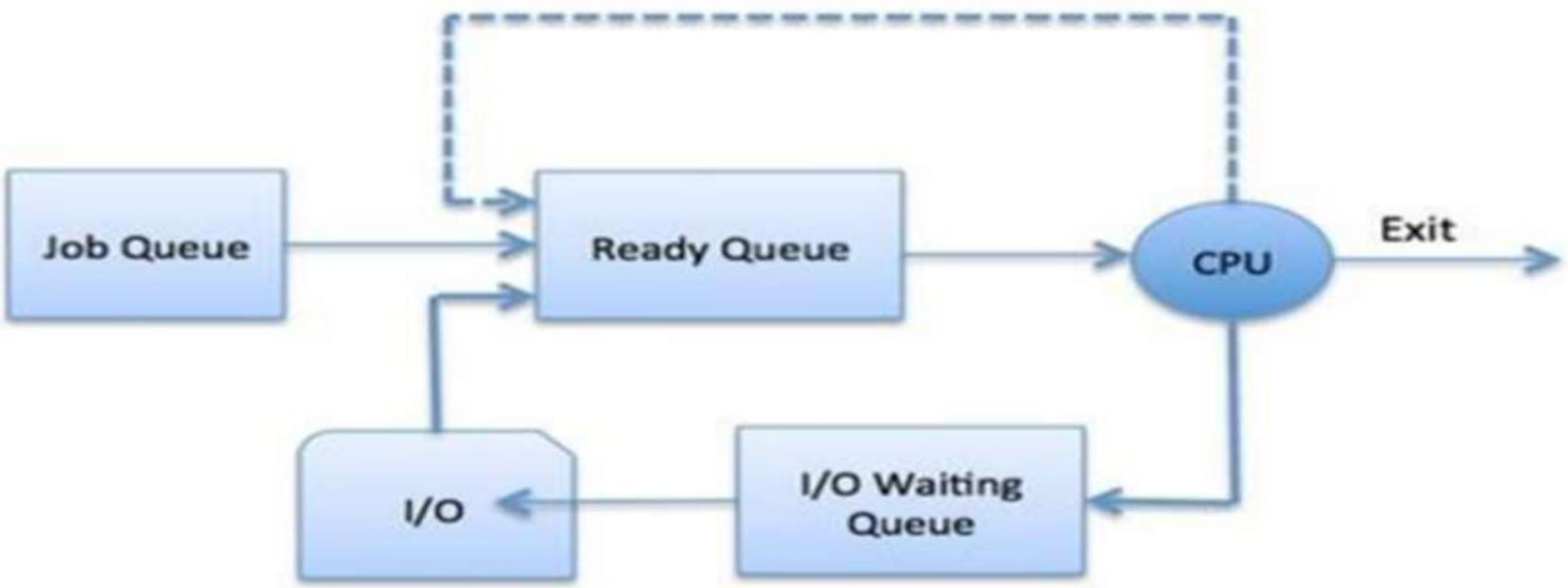
Waiting time:

- ✓ its the sum of periods waiting in the ready queue.

Response time:

- ✓ in interactive system the turnaround time is not the best criteria.
- ✓ Response time is the amount of time it takes to start responding, not the time taken to output that response.

Process Scheduling Queues



- The OS maintains all PCBs in Process Scheduling Queues.
- The OS maintains a separate queue for each of the process states and PCBs of all processes in the same execution state are placed in the same queue.
- When the state of a process is changed, its PCB is unlinked from its current queue and moved to its new state queue.
- The Operating System maintains the following important process scheduling queues −

✓ **Job queue** − This queue keeps all the processes in the system.

✓ **Ready queue** − This queue keeps a set of all processes residing in main memory, ready and waiting to execute. A new process is always put in this queue.

✓ **Device queues** − The processes which are blocked due to unavailability of an I/O device constitute this queue.

Two-State Process Model

Two -state process model refers to running and non-running states which are described below −

- **Running**:
- ✓ When a new process is created, it enters into the system as in the running state.
- **Not Running:**
- ✓ Processes that are not running are kept in queue, waiting for their turn to execute.
- ✓ Each entry in the queue is a pointer to a particular process.
- ✓ Queue is implemented by using linked list.
- ✓ Use of dispatcher is as follows.
- ✓ When a process is interrupted, that process is transferred in the waiting queue.
- ✓ If the process has completed or aborted, the process is discarded.
- ✓ In either case, the dispatcher then selects a process from the queue to execute.

Types of Scheduling:

❖ **Preemptive Scheduling:**

✓ **preemptive** scheduling algorithm picks a process and lets it run for a maximum of some fixed time

✓ If it is still running at the end of the time interval, it is suspended and the scheduler picks another process to run (if one is available).

- Doing preemptive scheduling requires having a clock interrupt occur at the end of time interval to give control **Non preemptive Scheduling**

❖ **Nonpreemptive** scheduling :

✓ **Nonpreemptive** scheduling algorithm picks a process to run and then just lets it run until it blocks (either on I/O or waiting for another process) or until it voluntarily releases the CPU.

✓ Even it runs for hours, it will not be forcibly suspended.

Preemptive Scheduling

- Preemptive scheduling is used when a process switches from running state to ready state or from waiting state to ready state.
- The resources (mainly CPU cycles) are allocated to the process for the limited amount of time and then is taken away, and the process is again placed back in the ready queue if that process still has CPU burst time remaining. T
- hat process stays in ready queue till it gets next chance to execute.
- Algorithms based on preemptive scheduling are: Round Robin (RR), Shortest Job First (SJF basically non preemptive) and Priority (non preemptive version), etc

Non-Preemptive Scheduling:

- Non-preemptive Scheduling is used when a process terminates, or a process switches from running to waiting state.
- In this scheduling, once the resources (CPU cycles) is allocated to a process, the process holds the CPU till it gets terminated or it reaches a waiting state.
- In case of non-preemptive scheduling does not interrupt a process running CPU in middle of the execution.
- Instead, it waits till the process complete its CPU burst time and then it can allocate the CPU to another process.
- Algorithms based on preemptive scheduling are: Shortest Remaining Time First (SRTF), Priority (preemptive version), etc.

PARAMENTER	PREEMPTIVE SCHEDULING	NON-PREEMPTIVE SCHEDULING
Basic	In this resources(CPU Cycle) are allocated to a process for a limited time.	Once resources(CPU Cycle) are allocated to a process, the process holds it till it completes its burst time or switches to waiting state.
Interrupt	Process can be interrupted in between.	Process can not be interrupted untill it terminates itself or its time is up.
Starvation	If a process having high priority frequently arrives in the ready queue, low priority process may starve.	If a process with long burst time is running CPU, then later coming process with less CPU burst time may starve.
Overhead	It has overheads of scheduling the processes.	It does not have overheads.
Flexibility	flexible	rigid
Cost	cost associated	no cost associated

Dispatcher

- A dispatcher is a special program which comes into play after the scheduler.
- When the scheduler completes its job of selecting a process, it is the dispatcher which takes that process to the desired state/queue.
- The dispatcher is the module that gives a process control over the CPU after it has been selected by the short-term scheduler.
- This function involves the following:
  - Switching context
  - Switching to user mode
- Jumping to the proper location in the user program to restart that program

Scheduler

- ✓ **Schedulers** are special system software which handle process scheduling in various ways.
- ✓ Their main task is to select the jobs to be submitted into the system and to decide which process to run.
- ✓ There are three types of Scheduler:
- **Long term (job) scheduler**
- **Medium term scheduler**
- **Short term (CPU) scheduler**

Long term (job) scheduler

- Due to the smaller size of main memory initially all program are stored in secondary memory.
- When they are stored or loaded in the main memory they are called process.
- This is the decision of long term scheduler that how many processes will stay in the ready queue.
- Hence, in simple words, long term scheduler decides the degree of multi-programming of system

Medium term scheduler

- Most often, a running process needs I/O operation which doesn't requires CPU.
- Hence during the execution of a process when a I/O operation is required then the operating system sends that process from running queue to blocked queue.
- When a process completes its I/O operation then it should again be shifted to ready queue.
- ALL these decisions are taken by the medium-term scheduler. Medium-term scheduling is a part of **swapping**.

Short term (CPU) scheduler

- When there are lots of processes in main memory initially all are present in the ready queue.
- Among all of the process, a single process is to be selected for execution.
- This decision is handled by short term scheduler.

Dispatcher vs Scheduler

PROPERTIES	DISPATCHER	SCHEDULER
Definition:	Dispatcher is a module that gives control of CPU to the process selected by short term scheduler	Scheduler is something which selects a process among various processes
Types:	There are no diifrent types in dispatcher.It is just a code segment.	There are 3 types of scheduler i.e. Long-term, Short-term, Medium-term
Dependency:	Working of dispatcher is dependednt on scheduler.Means dispatcher have to wait untill scheduler selects a process.	Scheduler works idependently.It works immediately when needed
Algorithm:	Dispatcher has no specific algorithm for its implementation	Scheduler works on various algorithm such as FCFS, SJF, RR etc.
Time Taken:	The time taken by dispatcher is called dispatch latency.	TIme taken by scheduler is usually negligible.Hence we neglect it.
Functions:	Dispatcher is also responsible for:Context Switching, Switch to user mode, Jumping to proper location when process again restarted	The only work of scheduler is selection of processes.

Scheduling Criteria

- There are several different criteria to consider when trying to select the "best" scheduling algorithm for a particular situation and environment, including:
- ✓ **CPU utilization** - Ideally the CPU would be busy 100% of the time, so as to waste 0 CPU cycles. On a real system CPU usage should range from 40% (lightly loaded) to 90% (heavily loaded.)
- ✓ **Throughput** - Number of processes completed per unit time. May range from 10 / second to 1 / hour depending on the specific processes.
- ✓ **Turnaround time** - Time required for a particular process to complete, from submission time to completion. (Wall clock time.)
- ✓ **Waiting time** - How much time processes spend in the ready queue waiting their turn to get on the CPU.
  - (**Load average** - The average number of processes sitting in the ready queue waiting their turn to get into the CPU. Reported in 1-minute, 5-minute, and 15-minute averages by "uptime" and "who".)
- ✓ **Response time** - The time taken in an interactive program from the issuance of a command to the ***commence*** of a response to that command.

What are the goals of CPU scheduling?

The goals of CPU scheduling are:

✓ Fairness:

- Each process gets fair share of the CPU.

✓ Efficiency:

- When CPU is 100% busy then efficiency is increased.

✓ Response Time:

- Minimize the response time for interactive user.

✓ Throughput:

- Maximizes jobs per given time period.

✓ Waiting Time:

- Minimizes total time spent waiting in the ready queue.

✓ Turn Around Time:

- Minimizes the time between submission and termination.

Types of CPU scheduling Algorithm

Scheduling

- Batch system scheduling
  - First come first served
  - Shortest job first
  - Shortest remaining time next
- Interactive System Scheduling
  - Round Robin scheduling
  - Priority scheduling
  - Multiple queues

1.First come first served

- ✓ FCFS is the simplest **non-preemptive** algorithm.
- ✓ Processes are assigned the CPU in the order they request it. That is the process that requests the CPU first is allocated the CPU first.
- ✓ The implementation of FCFS is policy is managed with a FIFO(First in first out) queue.
- ✓ When the first job enters the system from the outside in the morning, it is started immediately and allowed to run as long as it wants to.
- ✓ As other jobs come in, they are put onto the end of the queue.
- ✓ When the running process blocks, the first process on the queue is run next.
- ✓ When a blocked process becomes ready, like a newly arrived job, it is put on the end of the queue.

Advantages:

- Easy to understand and program.
- Equally fair.,
- Suitable specially for Batch Operating system.

Disadvantages:

FCFS is not suitable for time-sharing systems where it is important that each user should get the CPU for an equal amount of arrival time.

Disadvantages of FCFS

- The scheduling method is non preemptive, the process will run to the completion.
- Due to the non-preemptive nature of the algorithm, the problem of starvation may occur.
- Although it is easy to implement, but it is poor in performance since the average waiting time is higher as compare to other scheduling algorithms.

- **Arrival Time:** Time at which the process arrives in the ready queue.
- **Completion Time:** Time at which process completes its execution.
- **Burst Time:** Time required by a process for CPU execution.
- **Turn Around Time:** Time Difference between completion time and arrival time.

✓ Turn Around Time = Completion Time – Arrival Time

- **Waiting Time(W.T):** Time Difference between turn around time and burst time.

✓ Waiting Time = Turn Around Time – Burst Time

Q. Calculate the average waiting time if the processes arrive in the order of:

a). P1, P2, P3

b). P2, P3, P1

Process	Burst Time
P1	24
P2	3
P3	3

b.) If the process arrive in the order P2,P3, P1

- ✓ Average waiting time: (0+3+6)/3=3.
- ✓ Average waiting time vary substantially if the process CPU burst time vary greatly.

- ✓The processes arrive the order P1, P2, P3. Let us assume they arrive in the same time at 0 ms in the system.
- ✓We get the following gantt chart.

- ✓Waiting time for P1= 0ms , for P2 = 24 ms for P3 = 27ms
- ✓ Avg waiting time: (0+24+27)/3= 17

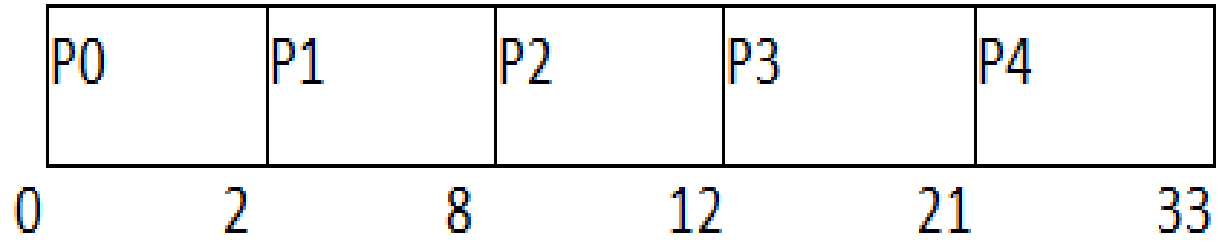
Example

In the Following schedule, there are 5 processes with process ID **P0, P1, P2, P3 and P4**. P0 arrives at time 0, P1 at time 1, P2 at time 2, P3 arrives at time 3 and Process P4 arrives at time 4 in the ready queue. The processes and their respective Arrival and Burst time are given in the following table.

- The Turnaround time and the waiting time are calculated by using the following formula
- ✓ Turn Around Time = Completion Time - Arrival Time
- ✓ Waiting Time = Turnaround time - Burst Time
- ✓ The average waiting Time is determined by summing the respective waiting time of all the processes and divided the sum by the total number of processes.

Process ID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
0	0	2	2	2	0
1	1	6	8	7	1
2	2	4	12	8	4
3	3	9	21	18	9
4	4	12	33	29	17

Avg Waiting Time=31/5



(Gantt chart)

Shortest Job First (SJF) Scheduling

- SJF scheduling algorithm, schedules the processes according to their burst time.
- In SJF scheduling, the process with the lowest burst time, among the list of available processes in the ready queue, is going to be scheduled next.
- However, it is very difficult to predict the burst time needed for a process hence this algorithm is very difficult to implement in the system.

Advantages :

- Maximum throughput
- Minimum average waiting and turnaround time

Disadvantages:

- May suffer with the problem of starvation
- It is not implementable because the exact Burst time for a process can't be known in advance

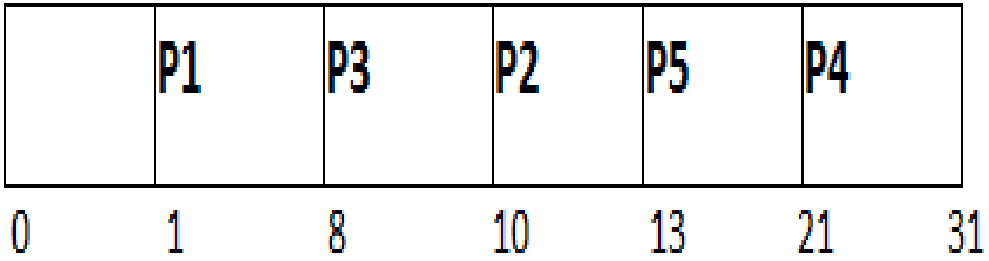
Example

- In the following example, there are five jobs named as P1, P2, P3, P4 and P5. Their arrival time and burst time are given in the table below.

PID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
1	1	7	8	7	0
2	3	3	13	10	7
3	6	2	10	4	2
4	7	10	31	24	14
5	9	8	21	12	4

- ✓ Since, No Process arrives at time 0 hence; there will be an empty slot in the **Gantt chart** from time 0 to 1 (the time at which the first process arrives).
- ✓ According to the algorithm, the OS schedules the process which is having the lowest burst time among the available processes in the ready queue.
- ✓ Till now, we have only one process in the ready queue hence the scheduler will schedule this to the processor no matter what is its burst time.
- ✓ This will be executed till 8 units of time.
- ✓ Till then we have three more processes arrived in the ready queue hence the scheduler will choose the process with the lowest burst time.
- ✓ Among the processes given in the table, P3 will be executed next since it is having the lowest burst time among all the available processes

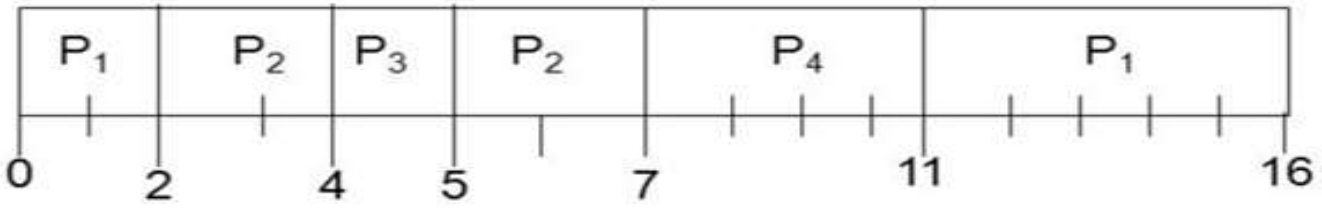
Gantt chart



Shortest remaining time next (SRTF)

- ✓ This Algorithm is the **preemptive version** of **SJF scheduling**.
- ✓ In SRTF, the execution of the process can be stopped after certain amount of time.
- ✓ At the arrival of every process, the short term scheduler schedules the process with the least remaining burst time among the list of available processes and the running process.
- ✓ Once all the processes are available in the **ready queue**, No preemption will be done and the algorithm will work as **SJF scheduling**.
- ✓ The context of the process is saved in the **Process Control Block** when the process is removed from the execution and the next process is scheduled. This PCB is accessed on the **next execution** of this process.

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4



- **Throughput**: 4 processes/16 milliseconds = 1/4
- **Turnaround time** for P_1 = 16-0 = 16; P_2 = 7-2 = 5; P_3 = 5-4 = 1; P_4 = 11-5 = 6
- **Average turnaround time**: (16 + 5 + 1 + 6)/4 = 28/4 = 7
- **Waiting time** for P_1 = 11-2 = 9; P_2 = 5-4 = 1; P_3 = 4-4 = 0; P_4 = 7-5 = 2
- **Average waiting time**: (9 + 1 + 0 +2)/4 = 12/4 = 3

Round-Robin Scheduling Algorithms:

- ✓ One of the oldest, simplest, fairest and most widely used algorithm is round robin (RR).
- ✓ In the round robin scheduling, processes are dispatched in a FIFO manner but are given a limited amount of CPU time called a time-slice or a quantum.
- ✓ If a process does not complete before its CPU-time expires, the CPU is preempted and given to the next process waiting in a queue.
- ✓ The preempted process is then placed at the back of the ready list.
- ✓ If the process has blocked or finished before the quantum has elapsed the CPU switching is done.
- ✓ Round Robin Scheduling is preemptive (at the end of time-slice) therefore it is effective in timesharing environments in which the system needs to guarantee reasonable response times for interactive users.

- The only interesting issue with round robin scheme is the length of the quantum.
- Setting the quantum too short causes too many context switches and lower the CPU efficiency.
- On the other hand, setting the quantum too long may cause poor response time and approximates FCFS.
- In any event, the average waiting time under round robin scheduling is on quite long.

Advantages:

- It can be actually implementable in the system because it is not depending on the burst time.
- It doesn't suffer from the problem of starvation or convoy effect.
- All the jobs get a fare allocation of CPU.

Disadvantages:

- The higher the time quantum, the higher the response time in the system.
- The lower the time quantum, the higher the context switching overhead in the system.
- Deciding a perfect time quantum is really a very difficult task in the system.

Process	Burst Time
P1	24
P2	3
P3	3

Quantum =4

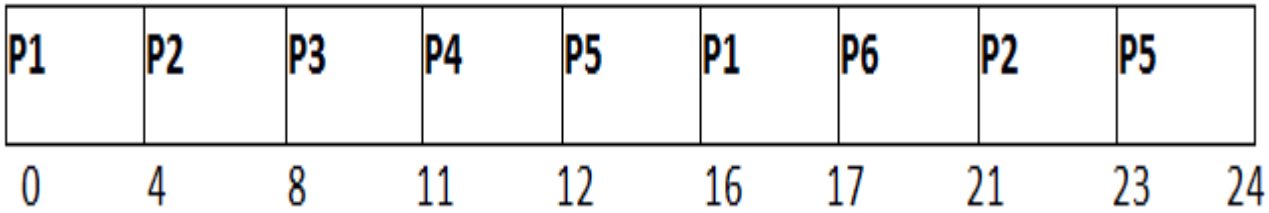
Solution :

P1	P2	P3	P1	P1	P1	P1	P1
0	4	7	10	14	18	22	26

RR Scheduling Example

- In the following example, there are six processes named as P1, P2, P3, P4, P5 and P6. Their arrival time and burst time are given below in the table. The time quantum of the system is 4 units.

Process ID	Arrival Time	Burst Time
1	0	5
2	1	6
3	2	3
4	3	1
5	4	5
6	6	4



The completion time, Turnaround time and waiting time will be calculated as shown in the table below.
As, we know,

1.Turn Around Time = Completion Time - Arrival Time

2.Waiting Time = Turn Around Time - Burst Time

Avg Waiting Time = (12+16+6+8+15+11)/6 = 76/6 units

Process ID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
1	0	5	17	17	12
2	1	6	23	22	16
3	2	3	11	9	6
4	3	1	12	9	8
5	4	5	24	20	15
6	6	4	21	15	11

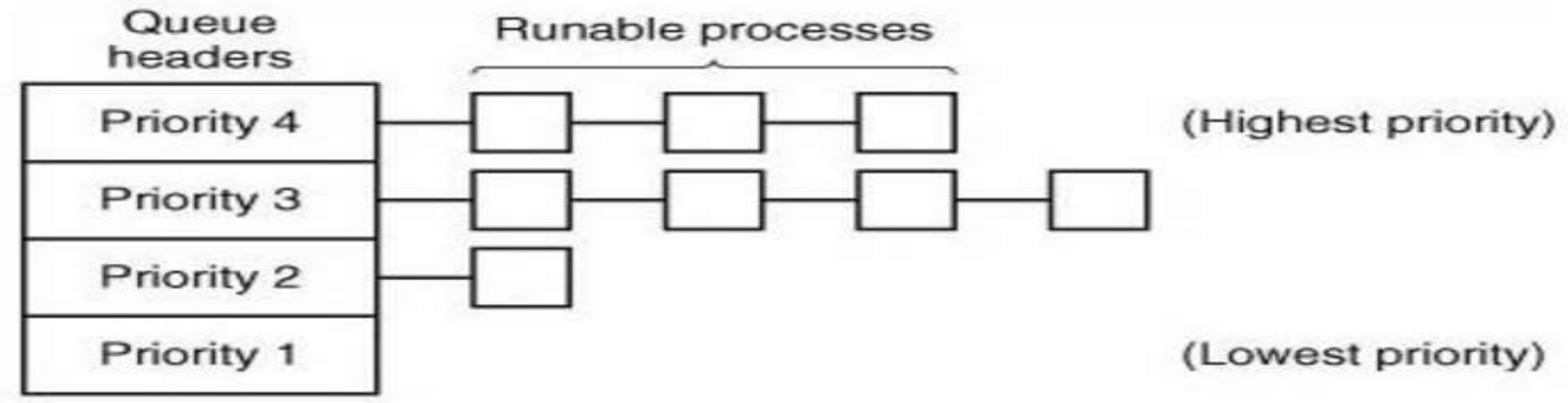
Priority Scheduling:

- A priority is associated with each process, and the CPU is allocated to the process with the highest priority.
- Equal priority processes are scheduled in the FCFS order.

Assigning priority:

- ✓ To prevent high priority process from running indefinitely the scheduler may decrease the priority of the currently running process at each clock interrupt.
- ✓ If this causes its priority to drop below that of the next highest process, a process switch occurs.
- ✓ Each process may be assigned a maximum time quantum that is allowed to run. When this quantum is used up, the next highest priority process is given a chance to run.

- It is often convenient to group processes into priority classes and use priority scheduling among the classes but round-robin scheduling within each class.



Examples

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

P2	P5	P1	P3	P4	
0	1	6	16	18	19

- ✓Let us look at an example of a multilevel queue scheduling algorithm with five queues, listed below in the order of priority.

1.System processes

2.Interactive processes

3.Interactive editing processes

4.Batch processes

5.Student processes

- Each queue has absolute priority over lower priority queues.